Third Year Project Report

Immersive NPC Interaction Design:

Bridging the Gap Between
Player Expectations & Industry Standards

Author: Dongmyung Park

Degree: BSc Artificial Intelligence

Student ID: 10879360

Supervisor: Gareth Henshall

Date: April 23, 2026

Abstract

Non-Player Characters (NPCs) in contemporary video games frequently suffer from context-blind behaviour and static relationships, undermining the player immersion that modern audiences demand. This report presents the research, design, and implementation of a context-aware NPC interaction system built in Unreal Engine 5.4.4, developed in direct response to player expectations identified through community feedback analysis on Reddit and a comparative study of industry titles including *Fable*, *Red Dead Redemption 2*, and *The Last of Us*.

Three recurring player pain points were identified: the absence of believable daily routines that make the game world feel persistently alive; context-blind dialogue that ignores the player's immediate physical and economic state; and the absence of an evolving relationship that reflects meaningful character growth over time. To address these, the prototype delivers six interdependent subsystems: a Behaviour Tree-driven daily schedule and patrol system with full spawn and despawn lifecycle management; a state-driven dialogue engine that prioritises stamina and financial warnings over standard greetings; a progressive trust mechanic that unlocks a Follow behaviour at a character-specific threshold and gates special item purchases at maximum trust; a probabilistic dialogue system allocating a 10% trigger rate for dynamic quests and 90% for contextual ambient interactions; a tiered item economy including procedurally generated rare items (10% spawn rate) coupled with a real-time trading interface; and a Mixamo-retargeted animation pipeline featuring stochastic idle blending governed by the Behaviour Tree.

Evaluation through functional testing, scenario-based integration tests, and probabilistic trial measurement ($N = 100$) confirms that the prototype effectively addresses all three failure modes, with stochastic subsystems producing outputs within one standard deviation of their designed 10% probability ($\sigma = 3.0$; observed rates: 11% and 14%). These results provide a replicable and data-driven foundation for immersive NPC design. A preliminary experiment with the `Qwen2.5-1.5B-Instruct` large language model revealed that base instruction-tuned models fail to respect game-specific personality and state constraints; future work therefore proposes domain-specific fine-tuning to enable context-aware, personality-driven NPC dialogue beyond the current authored pool.

Contents

1 Introduction **5**

1.1 Project Context 5

1.2 Problem Statement 5

1.3 Aims and Objectives 7

1.4 Report Structure 8

2 Background and Literature Review **10**

2.1 Evolution of NPC AI: From FSMs to Behaviour Trees 10

2.2 Case Studies in Commercial NPC Design 13

2.3 Dialogue Systems in Video Games 15

2.4 The Future of NPCs: Large Language Models and Generative Agents . . 16

3 Methodology and System Design **19**

3.1 Translating Expectations into Technical Requirements 19

3.2 Technical Framework and Iterative Prototyping 21

3.3 Core System Architecture 22

3.3.1 Player State and Environmental Inputs 22

3.3.2 NPC Logic Core 23

3.3.3 Data Management 23

3.4 Data-Driven NPC Design 24

3.5 Evaluation Methodology 26

4 Implementation **27**

4.1 World and Time Management System 27

4.1.1 24-Hour Cycle and Lighting Control 27

4.1.2 Dynamic Time-of-Day Audio System 29

4.1.3 Time-Synchronised NPC Lifecycle Management 30

4.2 Player Mechanics and Survival Systems 32

4.2.1 Stamina and Movement Dynamics 32

4.2.2 Perspective Shifting (1st/3rd Person) 34

4.3 NPC Artificial Intelligence and Animation Pipeline 35

4.3.1 Spatial Awareness and Pathfinding 35

4.3.2 Behaviour Tree Architecture and Stochastic Routines 37
4.3.3 Animation Pipeline and Embedded Auditory Feedback 39
4.4 Context-Aware Interaction System 40
4.4.1 Spatial Proximity and Intent Disambiguation 40
4.4.2 State-Driven Dialogue Priority Engine 43
4.4.3 Probabilistic Quests and Companion System 44
4.5 Economy, Trading, and Inventory Management 46
4.5.1 Data-Driven Inventory Architecture and Procedural Foraging . . . 46
4.5.2 Event-Driven Trading Interface and Contextual UX 48
4.5.3 Socio-Economic Coupling: Addressing Static Relationships 50
4.5.4 Trust-Gated Commerce and Special Modifiers 51

5 Evaluation and Testing **54**
5.1 Functional Unit Testing 54
5.1.1 Movement Speed Verification 55
5.1.2 State-Driven Dialogue Verification 56
5.1.3 Trust-Gated Follow Verification 57
5.1.4 Trust-Gated Commerce Verification 57
5.1.5 NPC Lifecycle Verification 58
5.2 Scenario-Based Integration Testing 58
5.2.1 Concurrent Depletion Scenario 59
5.2.2 Dense NPC Spatial Disambiguation 59
5.2.3 Temporal Boundary Interaction 60
5.2.4 End-to-End Relationship Progression 60
5.3 Probabilistic Trial Measurement 61
5.3.1 Quest Generation Rate 61
5.3.2 Rare Item Spawn Rate 62
5.4 Design Goal Evaluation 62
5.4.1 Failure Mode 1: Context-Blind Dialogue 63
5.4.2 Failure Mode 2: Absence of Believable Daily Routines 63
5.4.3 Failure Mode 3: Lack of Evolving Character Relationships 64
5.4.4 Summary of Evaluation 65
5.4.5 Threats to Validity 65

6 Conclusion and Future Work **67**

6.1 Project Summary 67

6.2 Limitations 68

6.3 Future Work 69

References **74**

Appendix A: External Resources and Asset Credits **77**

1 Introduction

1.1 Project Context

Video games have evolved from simple rule-based systems into expansive interactive worlds in which Non-Player Characters (NPCs) serve as the primary medium through which players experience narrative, economy, and social interaction. As Laird and van Lent [2] argued, the pursuit of human-level AI in interactive games represents one of the most compelling challenges in artificial intelligence, and one that remains largely unresolved today. Despite significant advances in graphics and world generation, the behavioural sophistication of NPCs has failed to keep pace with rising player expectations.

The importance of this gap extends beyond technical curiosity. Csikszentmihalyi's theory of Flow [10] established that deep engagement arises when a system responds meaningfully to a participant's actions. Sweetser and Wyeth [15] applied this to games through their GameFlow model, identifying NPC interaction as a key contributor to player enjoyment. Brown and Cairns [11] further demonstrated that immersion is actively undermined when game elements fail to meet player expectations — a failure mode that context-blind NPC behaviour directly exemplifies.

This project addresses this gap through a prototype developed in Unreal Engine 5.4.4 [1], delivering a suite of context-aware, data-driven NPC interaction mechanisms grounded in both academic literature and industry practice.

1.2 Problem Statement

Despite the central role NPCs play in player immersion, commercial titles continue to deploy interaction systems that suffer from three fundamental shortcomings, identified through community feedback analysis and a comparative study of industry titles.

To gather direct player perspectives, a purposive qualitative survey was submitted to the Reddit gaming community posing three questions: what makes an NPC unforgettable; what NPC behaviours break immersion; and whether players had witnessed surprising programmed behaviour. Responses from the community yielded consistent and revealing qualitative insights, as shown in Figures 1, 2, and 3.

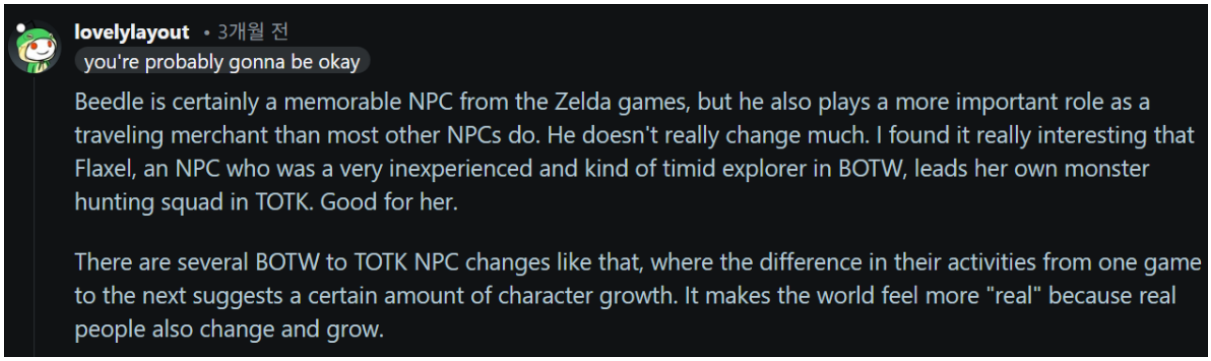


Figure 1: Response from Reddit user *lovelylayout*, highlighting character growth and evolving NPC behaviour as the primary driver of believability.

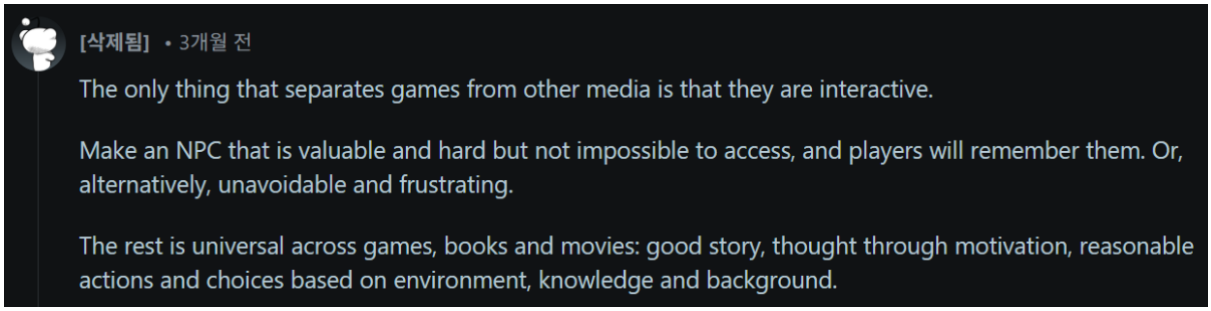


Figure 2: Response from an anonymous Reddit respondent, identifying interactivity and context-aware behaviour as the defining qualities of a memorable NPC.

Figure 3: Response from Reddit user *Khloe_trans*, emphasising emotional depth and vulnerability as the qualities that make an NPC feel like a real person.

One respondent highlighted character growth as the defining quality of believability, observing that NPCs in *The Legend of Zelda: Tears of the Kingdom* felt real precisely because their lives had visibly progressed since the previous instalment — "real people also change and grow." Another argued that interactivity is the quality unique to games, and that memorable NPCs must exhibit "reasonable actions and choices based on environment, knowledge and background." A further respondent identified emotional depth as the critical factor, noting that a single moment of vulnerability in *The Witcher 3* transformed Dandelion from comic relief into "a real, flawed person with dreams." Although qualitative community feedback is not statistically exhaustive, this purposive approach captures rich, experiential data that broad quantitative surveys often obscure. Importantly, these

anecdotal insights map onto established theoretical frameworks of player immersion [11, 12]. By triangulating this community feedback with academic literature, the project establishes a user-centric methodological foundation for its technical requirements.

Furthermore, the diversity of titles referenced by the respondents—ranging from systemic open-world exploration (*The Legend of Zelda*) to narrative-driven RPGs (*The Witcher 3*)—demonstrates that these NPC failure modes are not isolated to a single genre, but are systemic across the wider AAA games industry. This systemic nature is corroborated by a deeper historical analysis of industry benchmarks (explored comprehensively as Case Studies in Section 2.2). Briefly, the *Fable* series demonstrated the limits of a "wide but shallow" FSM approach [3]: NPCs reacted to global reputation scores but lacked individual depth. *Red Dead Redemption 2* addressed this through a large-scale Behaviour Tree (BT) architecture [5] governing NPCs with individual schedules. *The Last of Us* showed that narrative-driven rules could produce emotional resonance through deliberate constraint [20].

Synthesising these findings, three recurring failure modes emerge in existing NPC design: the absence of believable daily routines that make the game world feel persistently real [12]; context-blind dialogue that ignores the player's immediate physical and economic state [16]; and the lack of an evolving relationship mechanic that reflects meaningful character growth over time [19]. This project is designed specifically to address all three.

1.3 Aims and Objectives

The primary aim of this project is to design and implement a context-aware NPC interaction prototype in Unreal Engine 5.4.4 that demonstrably addresses the three failure modes identified in Section 1.2. The following objectives define the scope of the work:

1. **Investigate industry standards and player expectations** through a review of academic literature on game AI and player immersion [7, 13], a comparative analysis of commercial titles, and direct community feedback analysis.

2. **Design a data-driven NPC architecture** in which each character's behaviour, dialogue, and schedule are defined through structured data tables, enabling modular configuration without changes to the underlying logic [24].

3. **Implement a Behaviour Tree-driven daily schedule system** that governs NPC spawn and despawn lifecycle events, patrol behaviour, and context-sensitive off-duty states [5].

4. **Implement a state-driven dialogue and trust system** in which NPC responses are dynamically prioritised based on the player's stamina and financial state, and in which repeated interaction progressively unlocks new behaviours and items [19, 21].

5. **Implement a tiered economy and inventory system** featuring probabilistic item acquisition, a real-time trading interface, and trust-gated special items [22, 26].

The success of the project will be evaluated through a structured tripartite methodology—encompassing systematic functional testing, scenario-based integration testing, and probabilistic trial measurement [31]—which is formally outlined in Chapter 3 and executed in Chapter 5. This evaluation will map the quantitative and qualitative outcomes directly against the three failure modes established in Section 1.2.

1.4 Report Structure

The remainder of this report is organised as follows.

Chapter 2 provides a background and literature review, tracing the evolution of NPC AI from Finite State Machines to Behaviour Trees, examining dialogue system design, conducting case studies on commercial industry titles, and surveying the emerging role of Large Language Models in future NPC development.

Chapter 3 details the methodology and system design, covering the analysis of player expectations, the high-level system architecture, the data-driven design principles adopted throughout the project, and the formal evaluation methodology utilized for testing.

Chapter 4 presents the full implementation of the core subsystems: the world and time management system; player mechanics and survival systems; NPC artificial intelligence and daily schedules; the context-aware interaction system including dialogue priority and trust mechanics; and the economy, trading, and inventory management system.

Chapter 5 evaluates the implemented prototype through functional testing, scenario-based integration tests, and structured mapping against the three failure modes identified in Section 1.2.

Chapter 6 concludes the report with a summary of achievements, a critical reflection on the limitations of the prototype including a preliminary LLM experiment, and proposals for future work centred on domain-specific fine-tuning for context-aware NPC dialogue.

2 Background and Literature Review

2.1 Evolution of NPC AI: From FSMs to Behaviour Trees

The history of NPC artificial intelligence in video games is largely a history of managing complexity. Early game AI systems relied almost exclusively on hand-coded conditional logic and rule-based triggers, producing characters whose behaviour, while functional, was entirely predictable and inflexible. As games grew in scale and player expectations matured, the limitations of these approaches became increasingly apparent, driving a gradual but fundamental shift in how game developers architect NPC behaviour.

The Finite State Machine (FSM) represents the foundational paradigm of game AI, and one that dominated commercial development for several decades. In an FSM, an NPC exists in one of a finite number of predefined states — such as Idle, Patrol, Chase, or Attack — and transitions between them according to a fixed set of rules triggered by environmental conditions or player input [7]. The appeal of FSMs lies in their simplicity: they are easy to implement, easy to debug, and computationally inexpensive. For games of limited scope, this predictability is a feature rather than a limitation. Laird and van Lent [2] noted, however, that as the interactive complexity of games increased, the rigidity of FSM-based architectures began to produce behaviour that felt artificial and brittle — characters that could not gracefully handle situations their designers had not explicitly anticipated.

The limitations of FSMs were brought into sharp relief at the industrial scale by Isla's [3] account of the AI architecture underlying *Halo 2*. Isla described how the sheer number of states and transitions required to govern believable combat behaviour across multiple enemy types produced an FSM of such complexity that it became practically unmanageable. The state graph had grown so dense that adding new behaviours risked introducing unforeseen interactions between existing states, and the cognitive overhead of maintaining the system threatened to outpace the capacity of the development team. This account established a widely cited benchmark for the scalability ceiling of FSM-based game AI.

A complementary perspective was offered by Orkin [4] in his analysis of the AI system developed for *F.E.A.R.*, in which he demonstrated that goal-oriented action planning could replace large portions of hand-authored FSM logic with a more generalised decision-

making architecture. Rather than enumerating every possible state transition, the system allowed NPCs to reason about their objectives and select actions dynamically. This shift from reactive state management to deliberative planning represented a conceptual turning point in game AI design, illustrating that believable NPC behaviour need not be fully anticipated by the designer.

It was within this context that Behaviour Trees (BTs) emerged as the dominant architectural paradigm for commercial game AI. As Colledanchise and Ögren [5] define them, Behaviour Trees are hierarchical control structures in which composite nodes — Sequences, Selectors, and Parallels — govern the execution of leaf nodes representing atomic actions or conditions. The tree is ticked from root to leaf at each update cycle, and the result of each node — Success, Failure, or Running — propagates upward to determine the overall behaviour of the agent. This architecture offers several decisive advantages over FSMs for complex NPC systems: modularity, in that subtrees can be designed, tested, and reused independently; readability, in that the hierarchical structure reflects the logical organisation of behaviour in a way that flat state graphs do not; and scalability, in that new behaviours can be grafted onto existing trees without disrupting established logic. As illustrated in Figure 4, this hierarchical approach directly addresses the entanglement issues inherent in complex FSMs.

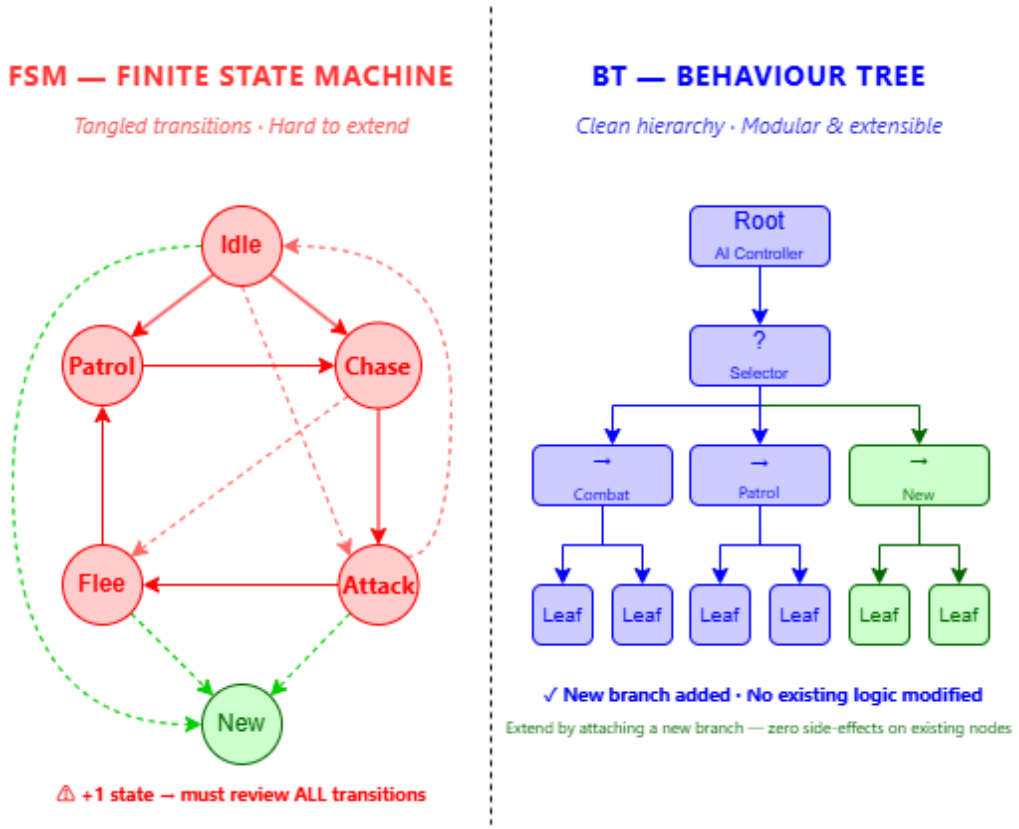


Figure 4: Structural comparison: The entangled state transitions of a Finite State Machine (left) versus the modular, hierarchical execution flow of a Behaviour Tree (right).

The breadth of BT adoption across both robotics and game AI is documented comprehensively by Iovino et al. [6], whose survey of over one hundred BT implementations confirms that the architecture has become the standard approach for autonomous agent control in domains requiring reactive, modular, and interpretable behaviour. In the game industry specifically, BTs now underpin the NPC systems of many major commercial titles, from the individual characters of *Halo 5* to the thousands of ambient agents populating the world of *Red Dead Redemption 2*. Yannakakis and Togelius [8] situate this trajectory within the broader evolution of game AI, arguing that the field has progressively moved from scripted reactivity towards architectures capable of exhibiting genuine contextual awareness.

A further consideration in NPC movement and spatial behaviour is the problem of pathfinding. The ability of an NPC to navigate a complex environment believably is a prerequisite for any higher-level behavioural system: a character whose path through the world is visibly mechanical or collision-prone will undermine immersion regardless of the sophistication of its decision-making. The A* algorithm, introduced by Hart, Nilsson, and

Raphael [9], remains the theoretical foundation of most real-time pathfinding systems in use today, including the Navigation Mesh (NavMesh) system provided by Unreal Engine 5 [1]. NavMesh-based pathfinding abstracts the traversable surface of an environment into a navigable polygon mesh, allowing agents to compute optimal paths efficiently at runtime. In the present project, this system underpins the patrol and schedule-driven movement of all NPCs.

However, spatial awareness alone is insufficient for believability without temporal coherence. Juul [17] conceptualises video games as being "half-real"—representing a duality between rigid, underlying mathematical rules and the fictional, simulated worlds they project. In the context of an NPC ecosystem, this duality is bridged primarily by the dimension of time. An agent that navigates a NavMesh effectively but stands idle in a town square 24 hours a day exposes the mechanical rules while failing the fictional simulation. Consequently, modern AI architectures must couple spatial pathfinding with robust global time-management systems to orchestrate believable daily schedules. This theoretical grounding serves as the architectural foundation for the global clock and lifecycle management systems detailed later in Chapters 3 and 4.

2.2 Case Studies in Commercial NPC Design

While the theoretical evolution from FSMs to BTs establishes the architectural foundation of modern game AI, the practical application of these systems in commercial titles highlights the specific methodological gaps this project aims to address. A comparative analysis of *Fable*, *Red Dead Redemption 2*, and *The Last of Us* illustrates how the industry has historically tackled the failure modes identified in Section 1.2, and where current limitations persist.

These three titles were selected as representative benchmarks because each addresses a distinct subset of the identified failure modes at industry-leading scale, yet none resolves all three simultaneously. The *Fable* franchise (Lionhead Studios) pioneered global reputation mechanics, attempting to create a responsive social ecosystem. However, constrained by early FSM architectures, the NPCs' reactions were predominantly broad and homogenous. Players experienced a "wide but shallow" social ecosystem [3]; crowds would universally cheer or boo based on a single global metric, but the game lacked granular, individu-

alised trust mechanics or economic depth, ultimately resulting in static and repetitive interpersonal relationships. This architectural limitation stems from the FSM's reliance on a single global variable as the sole input to all NPC state transitions, precluding the per-character parameterisation that a data-driven approach would afford [7].

Conversely, Rockstar Games' *Red Dead Redemption 2* represents a current industry benchmark for ambient NPC ecosystems. Utilizing massive, multi-layered Behaviour Trees, the game effectively addresses the "static presence" failure mode by assigning distinct daily routines, occupations, and spatial goals to thousands of agents. Nevertheless, despite their temporal believability, the interactions with ambient NPCs remain largely transactional and transient. The agents do not typically form evolving, long-term relationships with the player character based on repeated interactions, exposing a limitation in translating spatial autonomy into social depth. This suggests that spatial autonomy and social depth require fundamentally different architectural commitments: the former demands efficient scheduling algorithms, while the latter necessitates persistent, per-NPC relational state [24].

Finally, Naughty Dog's *The Last of Us* delivers highly context-aware and emotionally resonant NPC interactions, significantly reducing context-blind dialogue. Companion characters dynamically react to the player's health, ammunition scarcity, and environmental threats, surfacing critical survival states naturally. However, this level of contextual fidelity is achieved through rigorous, linear scripting and carefully orchestrated encounter design rather than systemic, open-world autonomy [20]. It demonstrates the value of state-driven priority logic but highlights the difficulty of scaling such bespoke reactivity into a dynamic ecosystem.

By analysing these commercial benchmarks, a clear methodological gap emerges. The industry has addressed these failure modes in isolation, but rarely in unison within a scalable, non-linear environment. Therefore, the architecture of this project is explicitly designed to synthesize the schedule-driven autonomy of *Red Dead Redemption 2* with the contextual reactivity of *The Last of Us*, underpinned by a granular, data-driven evolution of *Fable*'s reputation mechanics.

2.3 Dialogue Systems in Video Games

If the Behaviour Tree governs what an NPC does, the dialogue system governs what it says — and it is through dialogue that the player's most direct and personal interactions with NPC characters occur. The design of NPC dialogue systems has evolved in parallel with advances in behavioural AI, moving from static, menu-driven trees towards increasingly contextual and state-aware architectures.

The simplest and most widely deployed form of NPC dialogue is the branching dialogue tree, in which a fixed set of player choices leads to a fixed set of NPC responses, producing a conversation whose structure is fully determined at design time. While this approach affords writers precise narrative control, it scales poorly. A quintessential example of this limitation is observed in Bethesda's critically acclaimed *The Elder Scrolls V: Skyrim*. Despite its expansive open world, ambient NPCs rely on a shallow pool of state-triggered barks (e.g., the infamous "arrow in the knee" dialogue). When players hear the exact same rigidly authored line from dozens of visually distinct guards across different cities, the mechanical nature of the underlying tree is exposed, undermining the illusion of individual character depth. As the number of possible conversation states grows, the authoring burden increases combinatorially, and players frequently encounter the repetition and rigidity that the Reddit feedback analysed in Section 1.2 identified as primary sources of immersion breakdown. Isbister [20] argues that this rigidity is not merely a technical limitation but a psychological one: players form mental models of NPC characters through interaction, and when those models are violated by contextually inappropriate or highly repetitive responses, the illusion of a believable agent collapses.

A more sophisticated approach involves grounding dialogue selection in the runtime state of the game world. Rather than selecting responses from a fixed tree, a state-driven dialogue system evaluates a set of conditions — player proximity, inventory state, time of day, relationship history — and selects the most contextually appropriate response from a prioritised pool [16]. This approach, which underlies the dialogue architecture of titles such as *Red Dead Redemption 2*, produces conversations that feel responsive to the player's current situation rather than to a predetermined script. Bowman et al. [30] identify contextual responsiveness as a key determinant of perceived agent intelligence in interactive environments, noting that users consistently rate agents higher on measures of

believability when those agents demonstrate awareness of the user's immediate state. By proactively addressing the player's physiological or economic conditions, NPCs transition from static exposition dispensers to reactive participants in the game world.

The role of player motivation in shaping the design of dialogue systems is addressed by Yee [18], whose analysis of player motivations in online games identifies social interaction and relationship-building as primary drivers of engagement. Ryan, Rigby, and Przybylski [19] extend this analysis through the lens of Self-Determination Theory, arguing that the need for relatedness — the sense of meaningful connection with other agents in the environment — is a fundamental psychological need that games can satisfy through well-designed NPC interaction. These findings provide theoretical grounding for the trust-based relationship mechanic implemented in the present project, in which repeated positive interaction with an NPC progressively modifies its dialogue tone and unlocks new behavioural options.

Quest systems represent a specialised form of NPC-mediated dialogue in which conversation serves as the vehicle for goal assignment and narrative progression. Doran and Parberry [21] conducted a structural analysis of quest designs across four major MMORPGs, identifying a taxonomy of quest types — fetch, delivery, kill, escort, and others — that recur consistently across titles. Their analysis demonstrates that even simple quest structures, when delivered through contextually appropriate NPC dialogue, contribute meaningfully to player engagement and sense of agency. The present project implements a probabilistic quest system in which a 10% trigger rate governs whether an NPC interaction initiates a quest or produces ambient dialogue, drawing on this structural analysis to inform the design of the two quest types offered.

2.4 The Future of NPCs: Large Language Models and Generative Agents

The systems described in the preceding sections—Behaviour Trees, state-driven dialogue, trust mechanics—represent the current state of the art in structured NPC design. They are powerful, predictable, and practically deployable, but they share a fundamental constraint: every behaviour and every line of dialogue must be anticipated and authored in advance. No matter how sophisticated the branching logic, a player who exhausts the authored content will encounter repetition. The emerging application of Large Language Models

(LLMs) to NPC design represents a potential resolution to this constraint.

The most compelling demonstration of LLM-based NPC simulation to date is the Generative Agents system of Park et al. [32], in which twenty-five individually prompted LLM agents were placed in a simulated social environment and observed over an extended period. Each agent maintained a persistent memory stream of experiences, synthesised reflections from that memory, and used those reflections to plan future behaviour. The resulting agents exhibited emergent social behaviours—spontaneously organising a party, forming opinions about one another, spreading information through gossip—that had not been explicitly programmed. This work established that LLM-based agents are capable of sustaining the kind of evolving, context-sensitive behaviour that structured dialogue trees cannot produce.

From a game design perspective, the implications are significant. Schell [26] argues that the most memorable game experiences arise when players encounter systems that surprise them—when the game world responds in ways that feel both unexpected and coherent. An NPC powered by a generative language model, capable of responding to any player input with a contextually appropriate and narratively consistent reply, would represent a qualitative advance over any branching dialogue system. Shaker, Togelius, and Nelson [31] situate this possibility within the broader trajectory of procedural content generation, noting that the shift from authored to generated content is a defining trend in contemporary game design. Predictability here refers to the deterministic reproducibility of outputs given identical inputs—a property LLMs inherently lack due to temperature-based sampling. Table 1 summarises the fundamental trade-offs between these three paradigms, highlighting why Behaviour Trees currently offer the optimal balance for reliable game development.

AI Architecture	**Predictability**	**Scalability**	**Authoring Burden**
Finite State Machine (FSM)	High	Low	Very High
Behaviour Tree (BT)	High	High	Medium
Large Language Model (LLM)	Low	Very High	Low

Table 1: Comparison of NPC AI architectures. Categorisations synthesised from Isla [3], Colledanchise & Ögren [5], and Park et al. [32].

The present project does not implement LLM-based dialogue. While the computational

overhead and latency of real-time LLM inference remain prohibitive for a single-developer prototype, the primary deterrent is the risk to player immersion. Current LLMs often struggle to remain strictly confined to a specific historical era, fictional setting, or the immediate constraints of the game world. Without extensive guardrails, generative agents are prone to referencing out-of-world knowledge or producing dialogue that conflicts with the established lore, potentially undermining the constructed illusion of the game environment [17]. Consequently, the structured, probabilistic approach adopted here provides a more narratively cohesive and controlled basis for evaluating the specific interaction mechanics under investigation. Nevertheless, the integration of generative agents—properly constrained to maintain narrative integrity—is identified as a primary direction for future work, as detailed in Chapter 6.

3 Methodology and System Design

3.1 Translating Expectations into Technical Requirements

The fundamental challenge in developing immersive game AI lies in bridging the gap between qualitative player expectations and rigorous software engineering specifications [23]. As established in the problem statement (Section 1.2), the analysis of community feedback and industry benchmarks identified three primary failure modes in contemporary NPC design: context-blind interactions, static environmental presence, and the absence of meaningful relationship progression.

To address these deficiencies within the Unreal Engine 5 environment [1], the subjective player frustrations were abstracted and translated into three concrete, actionable technical pillars. These pillars encompass the six interdependent subsystems outlined in the project's scope, ensuring that every architectural decision serves a specific experiential goal. This maintains a modular and systemic design consistent with established game systems methodology [24] (detailed further in Chapter 4).

Firstly, to address the issue of *context-blind dialogue*, the system implements **State-Driven Priority Logic**. Rather than relying on traditional sequential dialogue trees, Blueprint logic evaluates the player's real-time variables — specifically stamina and currency — at the exact moment an interaction is triggered. Fagerholt and Lorentzon [16] demonstrate that surfacing critical player state information at the point of need, rather than requiring the player to consult a separate HUD element, significantly reduces cognitive interruption and maintains narrative immersion. If these variables fall below critical thresholds, the system overrides standard greetings, prompting the NPC to deliver a context-aware, urgent response first.

Secondly, the problem of *static environmental presence* necessitates a **Time-Synchronised AI Lifecycle** coupled with dynamic physical expression. A global time manager was engineered using a 1440-tick daily cycle, where one real-world second equates to one in-game minute. Governed by this clock, the AI Controller utilises a Behaviour Tree (BT) [5] to orchestrate the NPC's daily routine. Agents dynamically spawn into the world, execute their patrolling behaviours, and despawn to simulate retiring for the night. To prevent these routines from appearing mechanical, this is combined with a stochastic animation

pipeline that procedurally blends idle animations, creating a convincing temporal and physical rhythm consistent with systemic world-simulation [7].

Finally, to counteract the *lack of relationship growth*, the architecture relies on a **Data-Driven Trust System** that integrates with both quests and the economy. Managed via Unreal Engine's Data Tables, an integer-based trust metric dynamically scales the depth of available interactions [19]. As the player builds trust through a probabilistic quest system, the NPC's conversational tone progressively shifts. Furthermore, reaching specific trust thresholds integrates with a tiered item economy, unlocking advanced mechanics including a "Follow" command and access to an exclusive inventory of special items [21].

Table 2 summarises this methodological translation, mapping each identified experiential failure mode directly to its corresponding technical requirements and the six structural subsystems implemented in the prototype.

Identified Failure Mode (Sec 1.2)	**Derived Technical Requirement**	**Architectural Subsystems**
Context-blind dialogue ignoring player state	**State-Driven Priority Logic:** Blueprint scripting evaluates player stamina and wealth to trigger situation-appropriate dialogue before standard greetings [16].	1. State-Driven Dialogue Engine
Absence of believable daily routines	**Time-Synchronised Lifecycle:** A 1440-tick clock drives AI Controllers to spawn, work, and despawn via BTs, enhanced by stochastic physical expressions [5].	2. Behaviour Tree Schedule System 3. Stochastic Animation Pipeline
Lack of evolving character relationships	**Data-Driven Trust Ecosystem:** A Data Table metric gates interactions, driving procedural tasks and unlocking special economic items [19, 21].	4. Data-Driven Trust Mechanic 5. Probabilistic Quest System 6. Tiered Item Economy

Table 2: Mapping of qualitative player feedback to formal technical requirements, illustrating how the six implemented subsystems address the three primary failure modes.

By clearly defining these rules before implementation, the system ensures that complex contextual behaviours can be authored and scaled efficiently.

3.2 Technical Framework and Iterative Prototyping

To realise the requirements established in Section 3.1, Unreal Engine 5.4.4 (UE5) was selected as the primary development environment. The decision to utilise UE5's visual scripting system (Blueprints) and integrated Behaviour Tree (BT) framework over a purely C++ codebase was driven by two practical considerations: rapid iterative prototyping and runtime visual debugging [1].

Unlike traditional compiled languages where AI logic operates as a "black box" during execution, UE5's Blueprint framework allows for real-time visual tracing of data flow. This was essential for debugging complex state transitions, such as verifying that the stamina threshold reliably triggers the priority dialogue engine without bottlenecking the tick rate. Furthermore, the development lifecycle adhered to an *Iterative Prototyping* methodology. The numerical parameters that define the simulation's pacing—such as the 60-second stamina decay timer, the 2.0-second audio cross-fade, and the 10% to 33% stochastic animation probabilities (detailed comprehensively in Chapter 4)—were not arbitrarily assigned. They were derived through repeated empirical playtesting cycles designed to balance systemic friction with player immersion. By leveraging UE5's built-in Visual Logger and runtime AI debugging tools, the prototype's logic could be continuously profiled and refined to prevent framerate degradation (e.g., Garbage Collection spikes) during dense NPC interactions.

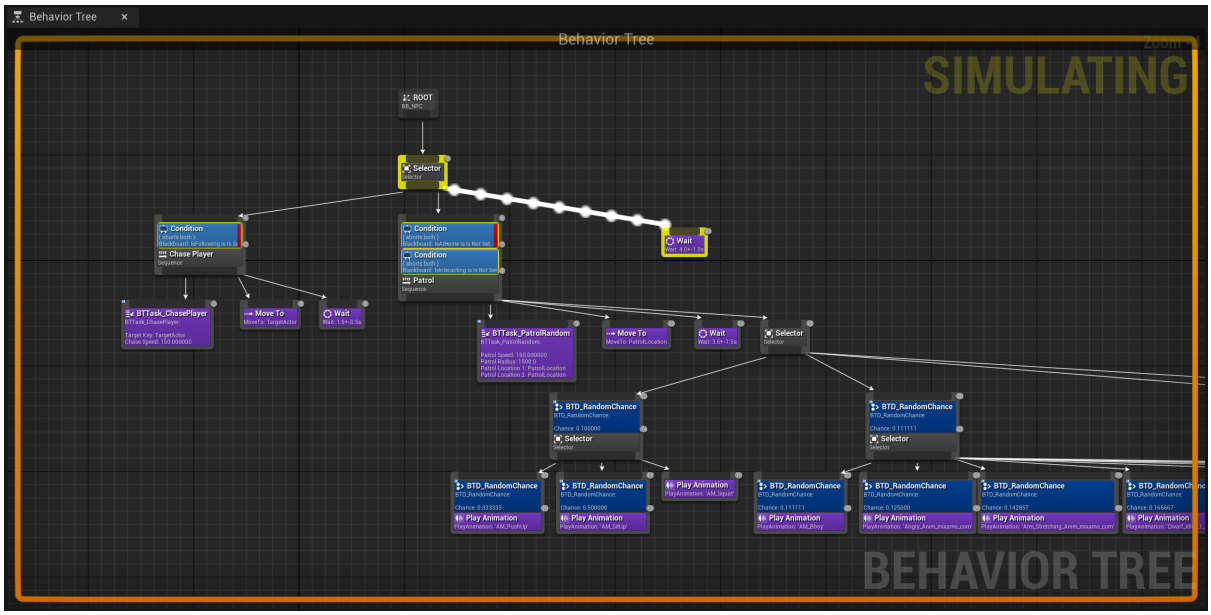


Figure 5: Runtime visual debugging within Unreal Engine 5, showing the active execution path (highlighted) of the Behaviour Tree during AI state transitions.

3.3 Core System Architecture

To satisfy the requirements established in Section 3.1, the NPC interaction system was engineered using a decoupled, modular architecture within Unreal Engine 5 [1]. The fundamental design philosophy isolates the real-time decision-making logic from static data storage, allowing for scalable character generation without redundant code duplication — a principle central to advanced game systems design [24].

Figure 6 illustrates the high-level data flow and component integration, which is structurally divided into three primary domains: Player State Management, the NPC Logic Core, and Data Management.

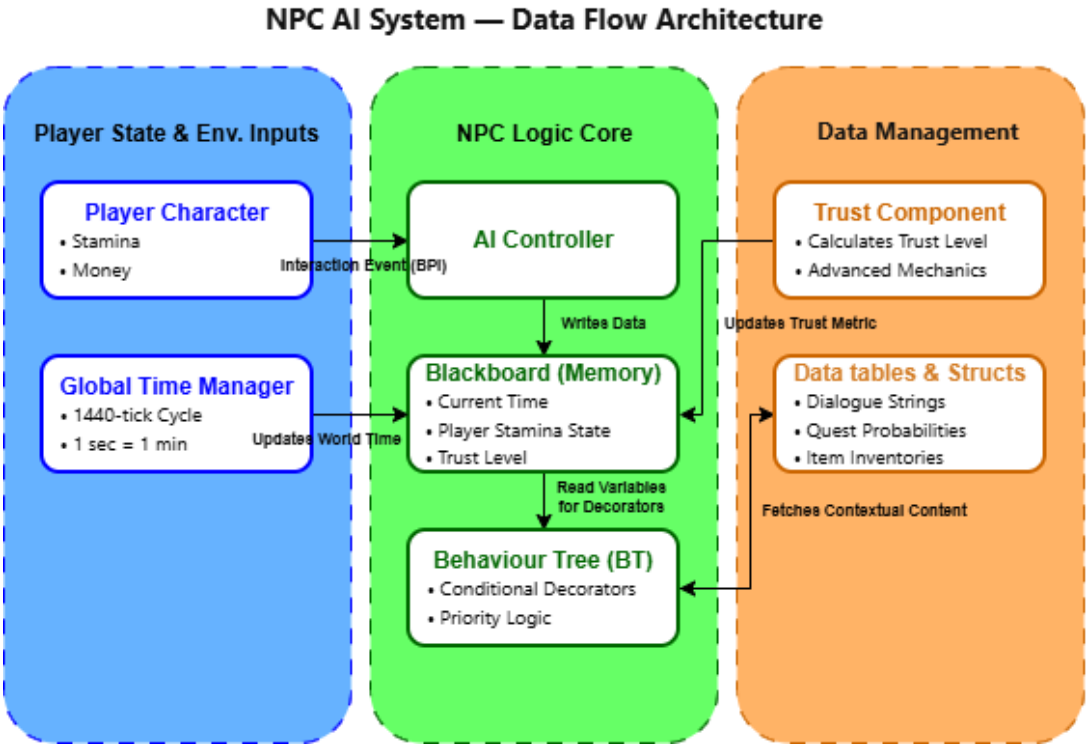


Figure 6: High-level System Architecture Diagram, illustrating the decoupled data flow between player state (Stamina/Money), AI logic, and data storage.

3.3.1 Player State and Environmental Inputs

This domain acts as the sensory input layer for the interaction system [30]. The **Player Character** continually monitors and broadcasts its own survival and economic variables, specifically *Stamina* and *Money*.

In terms of system architecture, the stamina metric dictates a multi-tiered response mechanism. Internally, it adjusts the player's physical capabilities, dynamically penalising

movement speed via the character movement component when dropping below a critical threshold of 20. Externally, it flags a state change when stamina falls below 30. This deliberate 10-unit differential serves as a key UX (User Experience) design choice: because stamina depletes at a fixed temporal rate, the higher threshold acts as a diegetic early-warning system, allowing NPCs to verbally warn the player one in-game hour before the severe physical penalty ensues (detailed further in Section 4.4.2). Upon initiating an interaction, this flag is transmitted via Blueprint Interfaces to the NPC, commanding the dialogue system to override standard greetings with exhaustion-specific responses, consistent with the state-driven design principles established by Fagerholt and Lorentzon [16]. The detailed Blueprint implementation of these mechanics is explored in Section 4.2.

Simultaneously, the **Global Time Manager** operates a 1440-tick background cycle (where one real-world second translates to one in-game minute), feeding continuous temporal data into the world state to govern NPC schedules. Juul [17] identifies the relationship between in-game time systems and player immersion as a critical design consideration, noting that coherent temporal structures reinforce the fictional reality of the game world.

3.3.2 NPC Logic Core

The Logic Core functions as the "brain" of the NPC, responsible for processing the aforementioned inputs and determining the appropriate output [7].

- **AI Controller & Blackboard:** The AI Controller acts as the operational bridge, intercepting player interaction events and updating the **Blackboard**. The Blackboard serves as the NPC's short-term memory, holding real-time evaluated values such as current time, passed stamina states, and the existing trust level [5].
- **Behaviour Tree (BT):** Driven by the data in the Blackboard, the BT acts as the primary decision engine [6]. It utilises conditional decorators to dynamically shift execution paths, allowing the AI to seamlessly transition between schedule-driven patrol routines and prioritised, state-driven player interactions.

3.3.3 Data Management

To maintain scalability, the actual content of the interactions is completely decoupled from the BT logic [23].

- **Data Tables & Structs:** All NPC-specific traits — including base trust requirements, dialogue strings for various emotional and physical states, and probability weightings for quest generation [31] — are stored externally in Unreal Engine Data Tables [1].

- **Trust Component:** This modular Blueprint component tracks the player's relationship with the NPC [20]. When specific trust thresholds are met, this component signals the logic core to fetch advanced data arrays from the Data Tables. This dynamic fetching directly updates the NPC's available interaction pool, seamlessly integrating the advanced mechanics (e.g., Follow state and gated inventories) established in Section 3.1 without altering the core Behaviour Tree structure.

By structuring the architecture in this manner, the system avoids hard-coded responses. Instead, the NPC dynamically pulls context-appropriate content from the Data Management layer based on the real-time player states processed by the Logic Core, realising the Mechanics-Dynamics-Aesthetics framework described by Hunicke, LeBlanc, and Zubek [25].

3.4 Data-Driven NPC Design

A fundamental principle of scalable game architecture is the separation of static content from runtime execution logic [24]. Hard-coding individual NPC behaviours, schedules, and thresholds directly into Blueprint scripts exponentially increases the authoring burden and introduces significant maintenance risks [23]. To achieve the scalability objective outlined in Section 3.1, this project employs a strictly data-driven design paradigm utilising Unreal Engine's Data Tables [1].

At the core of this system is a custom data structure (Struct) that acts as the master blueprint for every NPC in the game world. By defining character traits parametrically, designers can populate, balance, and iterate upon the game's social ecosystem without modifying the underlying Behaviour Tree logic [5]. Table 3 details the schema of this core NPC data structure, reflecting the exact parameters configured for the prototype's agents (Charles, George, Leon, Lucia, and Rose).

Variable Name	Data Type	Functional Description	Example (Charles)
`NPC_Name`	Text	The unique identifier and display name for the character.	"Charles"
`Personality_Type`	String	Defines the character's archetype (e.g., Busy, Grumpy, Friendly), acting as a key to fetch contextually appropriate dialogue lines [20].	"Busy"
`Start_Time` / `End_Time`	Float	Defines the active temporal window within the 1440-tick daily cycle (e.g., 420.0 to 1140.0), dictating when the BT triggers spawn and despawn behaviours [17].	420.0 / 1140.0
`Home_Location`	Vector (X,Y,Z)	Absolute world coordinates used by the NavMesh [9] for the NPC's default spawn point and resting destination.	(-2910, -6630, 200)
`Trust_Level`	Float	The baseline or initial trust metric allocated to the NPC upon instantiation [19].	0.0
`Required_Trust_Follow`	Float	The specific numerical threshold the player must reach to unlock the advanced "Follow" interaction mechanic [21].	80.0

Table 3: Schema of the core NPC Data Structure, including concrete sample data for the NPC 'Charles', demonstrating the parametrisation of schedules and interaction mechanics.

By centralising these parameters, the system's architecture becomes highly extensible [24]. For instance, the Behaviour Tree does not need to internally calculate *when* a specific NPC should sleep; it simply queries the `Start_Time` and `End_Time` variables from the Data Table and compares them against the Global Time Manager. Similarly, the system gates mechanics by comparing the player's runtime trust score against the `Required_Trust_Follow` variable.

3.5 Evaluation Methodology

As outlined in the project aims (Section 1.3), the success of the prototype is measured not solely by its functional compilation, but by its empirical effectiveness in addressing the three identified failure modes. To ensure a structured assessment, a tripartite evaluation methodology was designed, bridging the technical implementation in Chapter 4 with the formal evaluation in Chapter 5.

1. **Functional Unit Testing:** Before assessing the overarching player experience, the stability of individual subsystems must be verified. This involves controlled unit-level testing of specific Blueprint logic, such as confirming that the $O(N)$ search-and-stack algorithm correctly updates inventory variables without memory bloat, and ensuring that exhaustion variables (≤ 30) reliably override standard dialogue execution.

2. **Scenario-Based Integration Testing:** To evaluate the systemic coherence of the prototype, integration tests will subject the coupled mechanics to extreme edge-case player states. For example, initiating dialogue when the player concurrently possesses 0 stamina, 0 money, and an active quest. Furthermore, spatial integration will be tested by evaluating the reliability of the Dot Product vector calculations when interacting within dense crowds of overlapping NPC boundaries.

3. **Probabilistic Trial Measurement:** Procedural systems relying on Random Number Generation (RNG) are inherently non-deterministic. To empirically validate the reliability of the 10% stochastic triggers governing both dynamic quest generation and rare item procedural spawning, data will be gathered over a statistically significant sample size ($N \geq 100$ iterations).

This structured framework ensures that the subsequent evaluation (Chapter 5) is methodologically sound, confirming that the technical achievements directly address the qualitative player expectations established at the outset of this research.

4 Implementation

4.1 World and Time Management System

A fundamental prerequisite for a believable, schedule-driven NPC ecosystem is a globally consistent and stable time management system. To establish the foundational rhythm of the game world, a custom time progression architecture was implemented within the `BP_GameMode` using Unreal Engine's Blueprint visual scripting environment [1]. This system not only drives the visual day-night cycle but also serves as the master clock that dictates all contextual NPC behaviours, patrol routines, and economy shifts.

4.1.1 24-Hour Cycle and Lighting Control

Designing an appropriate time scale is a critical balancing act in open-world game development. Running the simulation in real-time (1:1 scale) often leads to pacing issues, where players experience prolonged periods of inactivity waiting for night-specific events. Conversely, a time cycle that is too rapid breaks immersion and makes NPC schedules appear erratic. To simulate a comprehensive 24-hour cycle while maintaining optimal player engagement, the temporal scale was accelerated such that one real-world second equates to one in-game minute. Consequently, a full virtual day is resolved within exactly 1440 seconds (24 minutes). This approach aligns with fundamental game design principles [22, 23] and bridges the duality between rigid game rules and the fictional representation of time [17].

At the core of this system is the `CurrentTime` float variable, updated every frame via the `Event Tick` node. To ensure long-term stability and prevent floating-point overflow during extended gameplay sessions, a Modulo operation is applied to the accumulated time. The progression is calculated as follows:

$$CurrentTime = (CurrentTime + (\Delta t \times TimeSpeed)) \bmod 1440$$

Where `TimeSpeed` is explicitly set to 1.0 within the Blueprint execution, and by clamping the resulting value to a maximum of 1440, the system seamlessly loops back to midnight (0.0) without requiring complex, performance-heavy reset logic. For this prototype, the `CurrentTime` is initialised with a default value of 360.0. Since 60 in-game minutes equal

one hour, 360 ticks correspond to 06:00 AM, providing an optimal morning starting condition for the player.

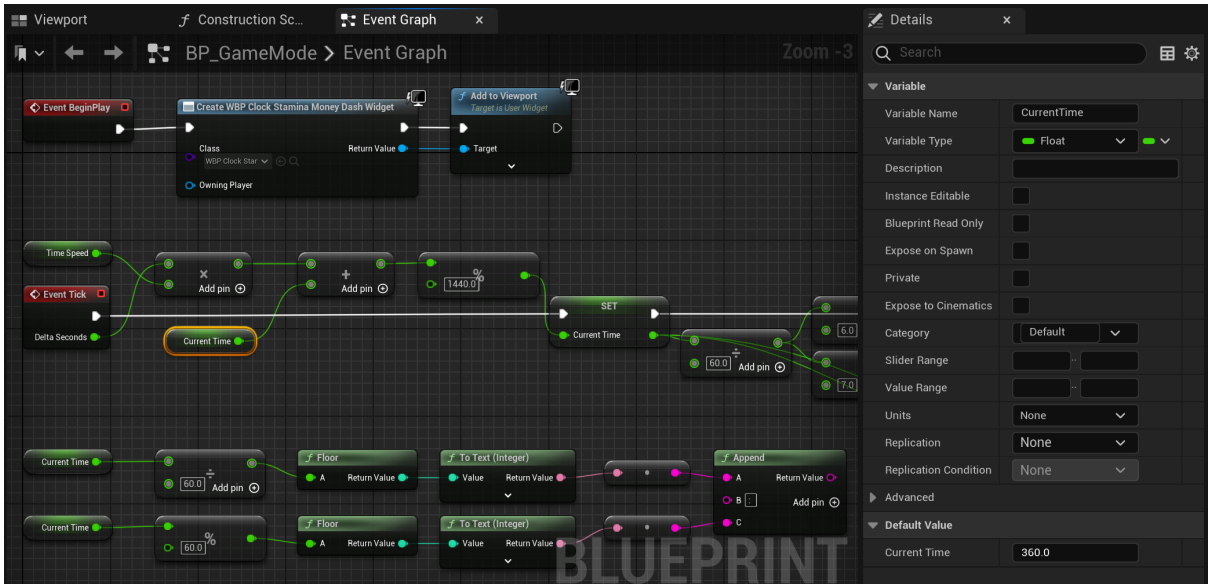


Figure 7: Blueprint implementation of the master clock, utilising Delta Seconds and Modulo (%) logic to maintain a continuous 1440-tick cycle without overflow.

To surface this temporal data to the player without breaking immersion, a decoupling logic was implemented for the User Interface. The raw float value is decomposed using floor division ($\div 60$) to extract the Hour, and a modulo operation ($\mathrm{mod} 60$) to extract the Minute. These integers are subsequently formatted into an `{Hour}:{Minute}` string via a `Format Text` node and broadcasted to the `WBP_Clock_Stamina` UI widget located at the top-left of the viewport, adhering to immersive HUD design guidelines that expose critical simulation data without obscuring gameplay [16].

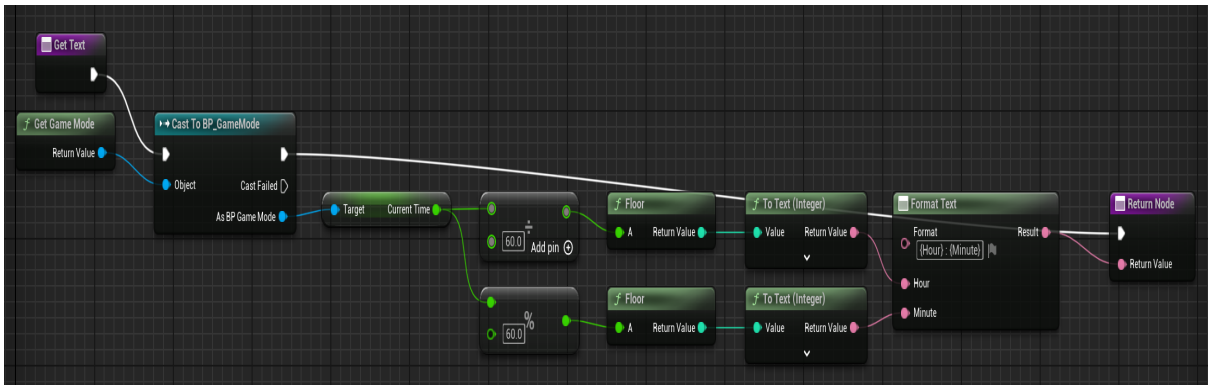


Figure 8: Decomposition of the raw float time into UI-friendly Hour and Minute integers using floor and modulo operations.

The aesthetic representation of time is driven by manipulating the rotational coordinates

of a `Directional Light` actor. Initially positioned in the global East with a Y-axis (Pitch) value of 0.0 (parallel to the horizon), the sun's rotation must map accurately to the 1440-tick timeline. To achieve this smoothly without introducing frame-rate-dependent stuttering, a dedicated `Timeline` node was utilised. The timeline maps the input time range of $0.0 \sim 1440.0$ to a rotational output of $0.0° \sim 360.0°$. Since the starting time is 06:00 AM (Pitch 0°), the sun naturally reaches its zenith at midday and sets at 18:00 PM, generating physically intuitive shadows across the NavMesh.

Furthermore, dynamic environmental lighting was integrated to enhance the world's believability. Modifying global illumination heavily impacts rendering performance; therefore, rather than updating emissive materials on every tick, the `Timeline` governs specific `EventOnDay` and `EventOnNight` execution pins as lightweight triggers. At 450 ticks (07:30 AM), the ambient house lights are scheduled to turn off. Conversely, at 1020 ticks (17:00 PM), the lights reactivate. Notably, the system linearly interpolates (Lerps) the emissive properties of the building meshes over a defined duration. This gradual transition between lit and unlit material states creates an immersive, "lived-in" atmosphere that hard binary toggling cannot replicate, effectively satisfying the environmental design goals established previously.

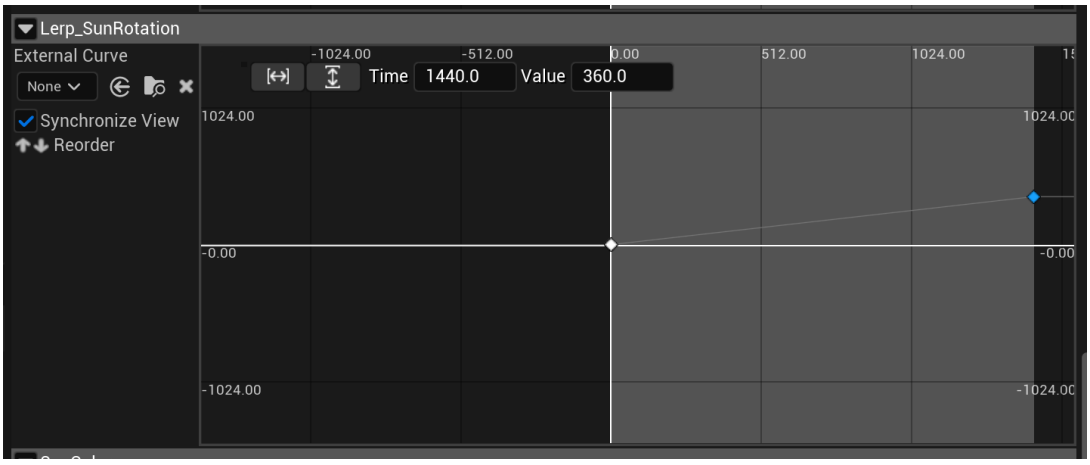


Figure 9: Timeline-driven mapping of the 1440-tick cycle to 360° Directional Light Pitch, including execution pins for environmental emissive updates.

4.1.2 Dynamic Time-of-Day Audio System

To further augment the immersive quality of the day-night cycle, a dynamic chronological audio system was implemented, directly tethered to the global `CurrentTime` variable. Visual environmental changes alone are often insufficient to sustain player immersion;

auditory feedback must parallel the visual state [12].

As the in-game time progresses through predefined thresholds (e.g., dawn, dusk, night), the system evaluates the current Hour using a series of `Branch` nodes coupled with `AND` Boolean logic. To ensure systemic optimization and prevent overlapping audio instances from being spawned on every frame tick, `DoOnce` nodes are utilized as execution gates.

Importantly, when a temporal state transition occurs, the system does not abruptly terminate the previous track. Instead, it validates the active audio component and invokes a programmatic `Fade Out` function over a 2.0-second duration. This specific temporal window was established through iterative playtesting and aligns with interactive audio design principles: a shorter fade produces a jarring, artificial cutoff, whereas a prolonged cross-fade risks dissonant auditory clutter during rapid environmental transitions. Subsequently, a `Spawn Sound 2D` node initializes the new ambient track, updating the active BGM reference. This auditory cross-fading prevents disruptive audio clipping, maintaining a seamless sensory continuum that closely mirrors the visual lighting transitions of the environment.

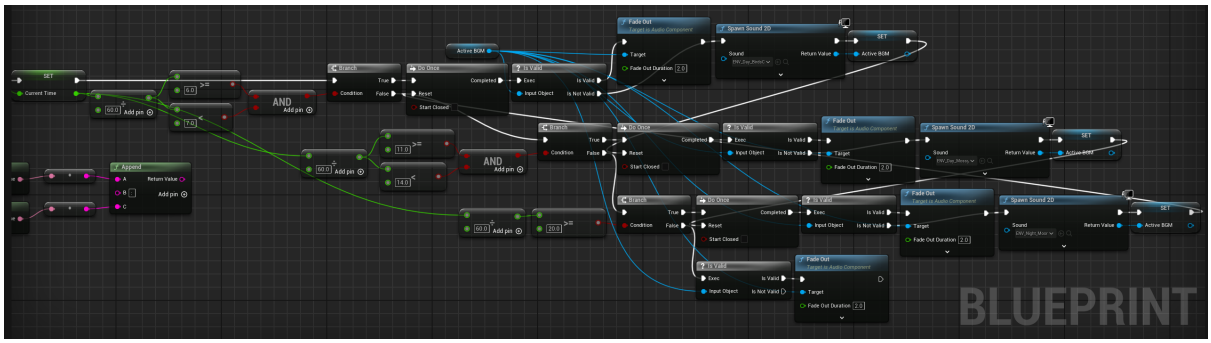


Figure 10: Blueprint implementation of the dynamic audio system, showing temporal Boolean logic, DoOnce execution gates, and 2.0-second cross-fading for BGM transitions.

4.1.3 Time-Synchronised NPC Lifecycle Management

A robust global clock is only as effective as the systemic reactions it produces within the game world. To directly address the immersion failure modes regarding static, unresponsive game environments (identified in Section 1.2), the temporal data from the master clock is deeply integrated into the physical lifecycle and memory management of the NPCs. This establishes a believable daily routine where characters dynamically commute to work and retire to their homes.

The logic governing this time-synchronised presence is detailed in Figure 11. Each NPC continually evaluates the global `CurrentTime` against their individually assigned operational window (defined parametrically in their Data Table, as discussed in Section 3.4). This is achieved via a Boolean `AND` gate, checking if the time is greater than or equal to their shift start, and less than their shift end.

If the temporal condition evaluates to True (the active working hours), the system updates the NPC's Blackboard memory, setting the `IsAtHome` key to False. Rather than utilizing computationally expensive `SpawnActor` functions, the system employs an optimized rendering approach. It invokes `Set Actor Hidden In Game` to False and `Set Actor Enable Collision` to True. This method mimics Object Pooling paradigms, preventing Garbage Collection (GC) spikes and maintaining framerate stability by keeping the actor in memory but toggling its physical and visual presence [24].

Conversely, when the temporal condition evaluates to False (the shift ends), the system executes an immersive despawn sequence. Instead of abruptly destroying the actor—a jarring event that undermines player immersion [20]—the Blackboard `IsAtHome` key is set to True, and an asynchronous `AI MoveTo` command is dispatched. The NPC actively navigates the NavMesh towards its designated `Home Location`. Only upon the `On Success` execution pin firing (signifying the NPC has physically reached its door) does the system re-engage the visibility and collision toggles to hide the character. This ensures that the illusion of a living, breathing village remains unbroken from the player's perspective.

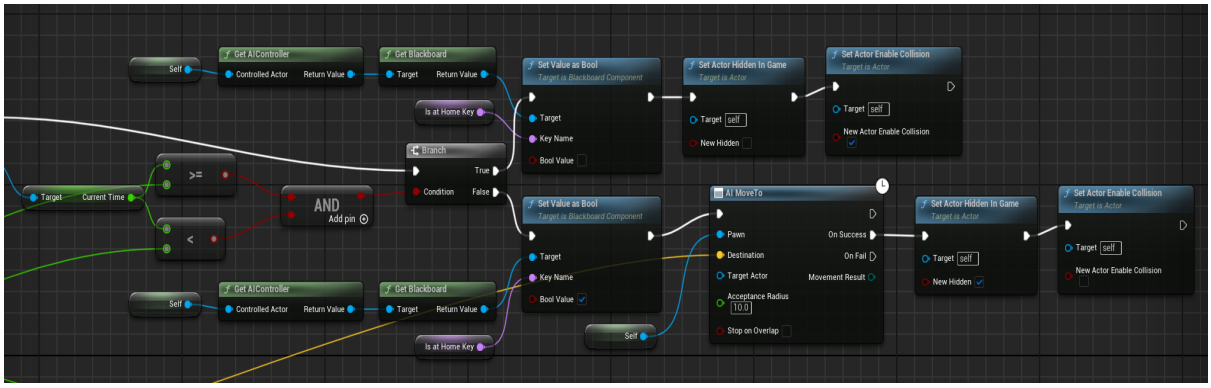


Figure 11: Blueprint implementation of the time-synchronised NPC lifecycle, showing asynchronous `AI MoveTo` for immersive commuting and collision/visibility toggling for memory-efficient despawning.

4.2 Player Mechanics and Survival Systems

Following the established framework for global simulation time (Section 4.1), the player character's mechanics were implemented to impose meaningful survival constraints. This section details the technical implementation of the stamina-based exhaustion system and the perspective shifting mechanism, which serve as core difficulty drivers within the prototyped core loop [22].

4.2.1 Stamina and Movement Dynamics

The core of the survival simulation is the stamina system, designed to enforce a physiological constraint on player exploration and encourage engagement with the NPC economy for resources. The implementation adopts a system dynamics approach where stamina is a finite stock variable subject to continuous depletion.

As illustrated in Figure 12, the `BP_ThirdPersonCharacter` utilizes a looping `Set Timer by Event` node, initialized at `Event BeginPlay`. For this prototype, the timer is configured with a frequency of 60.0 seconds to simulate passive long-term fatigue (e.g., hunger). This specific duration was selected based on iterative playtesting to establish a balanced core loop, ensuring players feel a persistent survival pressure without being overly burdened by hyperactive stat degradation. It can also easily be bound to movement input (e.g., sprinting) for active decay. The triggered custom event, `DecreaseStaminaOverTime`, calculates the new stamina value by subtracting a predefined `ReduceStamina` float from the `CurrentStamina`. To maintain numerical stability and ensure data integrity, a `Clamp (Float)` node is utilized to restrict the resulting value between a minimum of 0.0 and a maximum of 100.0 (`MaxStamina`), preventing floating-point underflow or invalid state transitions.

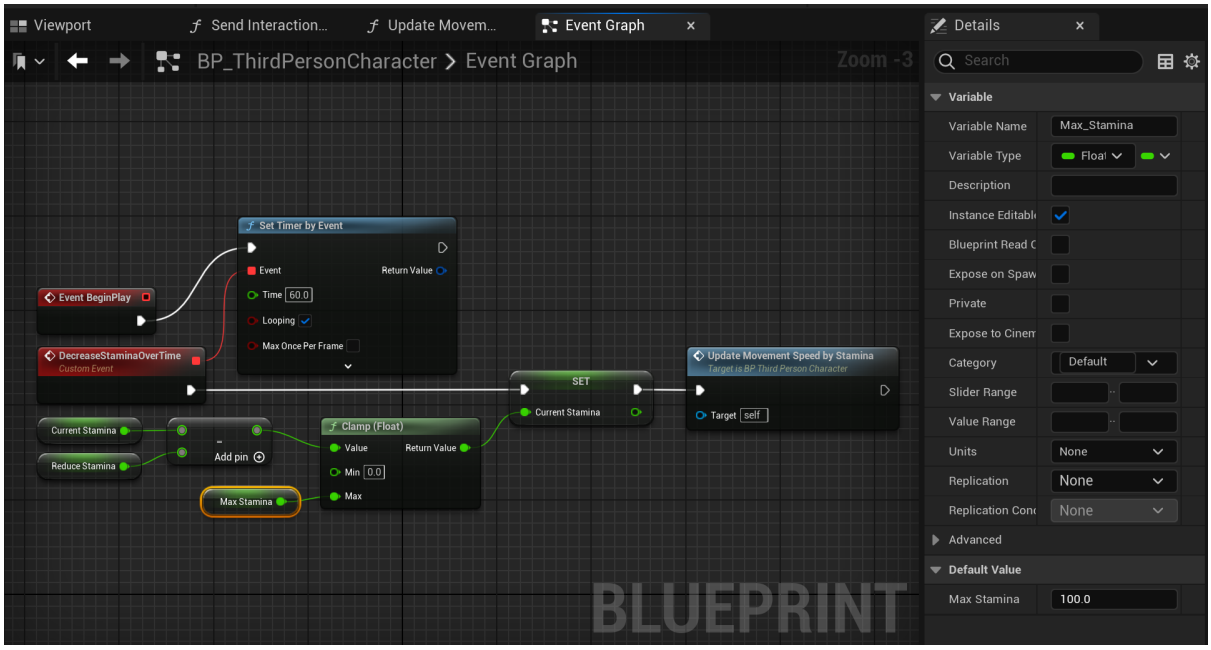


Figure 12: Blueprint implementation of passive stamina decay utilizing a 60-second looping timer and Clamp logic for numerical stability.

The functional manifestation of fatigue is implemented by coupling the numerical stamina state to the `CharacterMovementComponent`. Every time stamina is updated, a dedicated function, `UpdateMovementSpeedbyStamina`, is called. Figure 13 shows the logical architecture of this coupling. A `Branch` node serves as the decision gate, checking a specific threshold condition: `CurrentStamina < 20.0`.

If this fatigue threshold is met (True path), the system assumes an exhausted state. It accesses the `CharacterMovementComponent` and overrides the `MaxWalkSpeed` to a significantly lower value of 200.0. If the stamina remains above 20.0 (False path), the system maintains the player in a healthy state, setting the speed back to the default `BaseSpeed` of 300.0. This binary state mapping enforces a distinct physical penalty for poor stamina management, satisfying the game design requirement for immediate and legible feedback regarding survival state transitions.

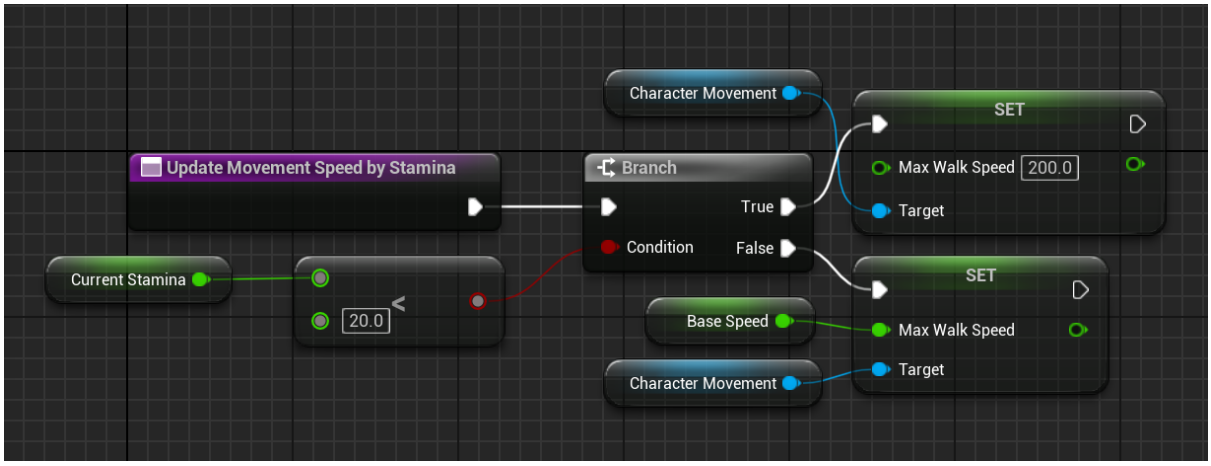


Figure 13: Fatigue penalty logical flow, mapping the stamina threshold (< 20.0) to the CharacterMovement's MaxWalkSpeed.

4.2.2 Perspective Shifting (1st/3rd Person)

To accommodate varying player preferences for spatial navigation, a perspective shifting mechanism was implemented, mapped to the 'V' key Action Input [14]. By allowing players to transition from broad third-person environmental navigation to face-to-face first-person dialogue framing, the perspective shift complements the core interaction systems and reinforces visual immersion during NPC conversations.

The technical pipeline is detailed in Figure 14. The 'V' key triggers a `FlipFlop` node, managing a binary state machine for perspective.

On the initial trigger (Path A: First-Person Transition):

1. The `CameraBoom`'s `TargetArmLength` is set to 0.0.
2. A `SocketOffset` of $(X = 0.0, Y = 0.0, Z = 80.0)$ is applied to elevate the camera to the character model's eye-level.
3. To resolve mesh clipping (players seeing the inside of their own model), `HideBonebyName` is called on the skeletal `Mesh` for the 'head' bone.

On the subsequent trigger (Path B: Third-Person Restoration):

1. The `TargetArmLength` is restored to a default of 400.0.
2. The `SocketOffset` is reset to 0.0.

3. The head bone is restored via `UnHideBonebyName`.

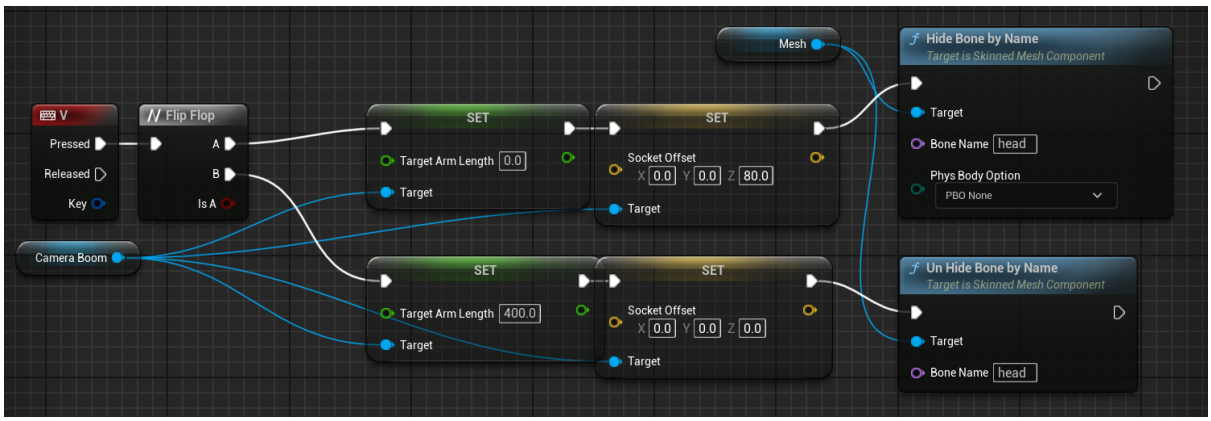


Figure 14: Perspective toggling logic using a FlipFlop state machine, including Spring Arm adjustment and head bone visibility manipulation to prevent camera clipping.

4.3 NPC Artificial Intelligence and Animation Pipeline

The cornerstone of a believable open-world ecosystem lies in the autonomy and behavioural complexity of its Non-Playable Characters (NPCs). Traditional Finite State Machines (FSMs) often suffer from scalability issues when managing complex, multi-layered daily routines. Consequently, this project adopted a Behaviour Tree (BT) architecture [5, 6], integrated with dynamic spatial pathfinding and a customized animation blending pipeline, to create an immersive NPC ecosystem.

4.3.1 Spatial Awareness and Pathfinding

For NPCs to interact meaningfully with the environment, they require an understanding of navigable space. This was achieved by overlaying the landscape with a `RecastNavMesh-Default` Bounds Volume. The NavMesh algorithm pre-calculates the geometry of the terrain, dividing it into traversable convex polygons and applying A* pathfinding algorithms [9, 7] to ensure collision-free movement across complex topology.

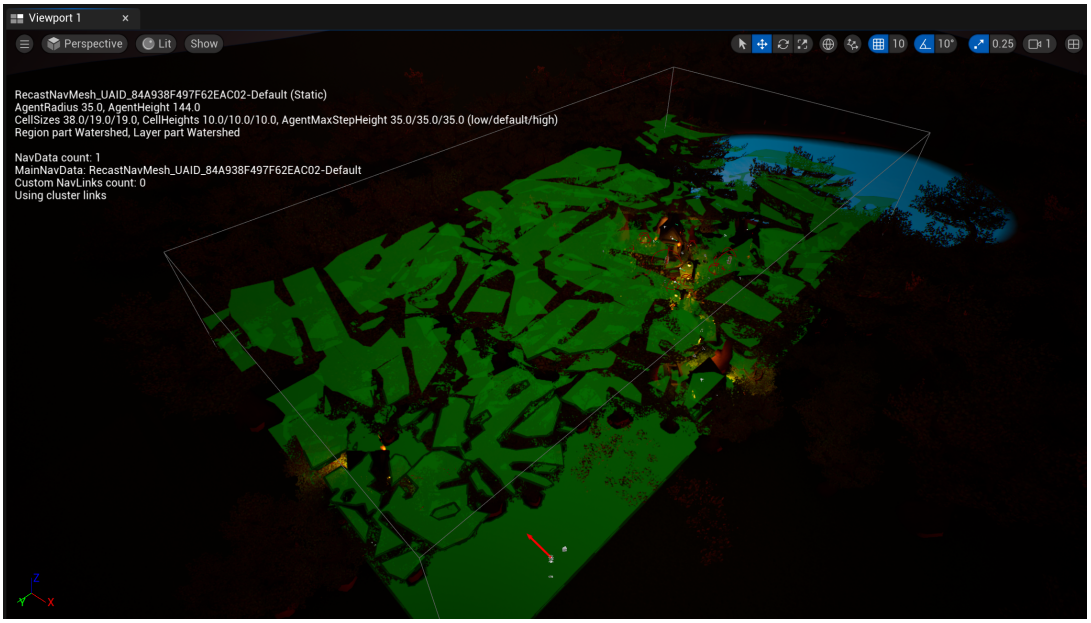


Figure 15: Recast NavMesh Bounds Volume (visualised in green), defining the navigable spatial boundaries for NPC pathfinding.

Movement logic was modularised into custom BT Tasks. As illustrated in Figure 16, the `BTTask_PatrolRandom` blueprint acts as the primary exploration driver. Upon receiving the `Execute AI` event, the logic casts to the `BP_NPC_Standard` controller and updates the `MaxWalkSpeed` to a dedicated `PatrolSpeed`. It utilises the `GetRandomReachablePointInRadius` node, querying the NavData to dynamically select a valid Cartesian coordinate within a defined radius of 1500.0 units. This coordinate is then written to the Blackboard as a Vector key (`PatrolLocation`), ensuring the NPC organically traverses the NavMesh rather than following rigid, pre-determined splines. A parallel task, `BTTask_ChasePlayer`, dynamically updates the target vector to the player's real-time location and increases the movement speed when hostile or follower conditions are met.

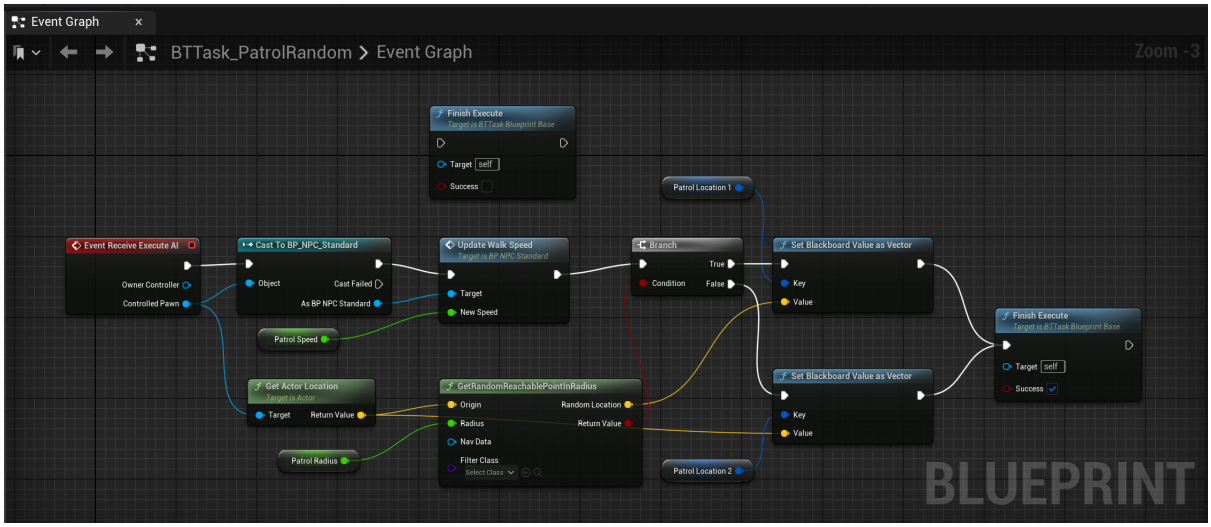


Figure 16: Internal logic of the custom Patrol Task, querying the NavMesh for a random reachable coordinate and updating Blackboard memory vectors.

4.3.2 Behaviour Tree Architecture and Stochastic Routines

The decision-making brain of the NPC is constructed within the `BB_NPC` Behaviour Tree. The hierarchical structure begins at the `ROOT`, filtering through a primary `Selector` node that evaluates left-to-right priority branches based on Blackboard conditions [3]. If the `IsFollowing` Boolean is set, the sequence triggers the Chase logic. The subsequent branch evaluates the `IsAtHome` condition; as established in Section 4.1.3, this Blackboard key is dynamically injected by the master clock's lifecycle manager. If the NPC is not interacting (`IsInteracting is Not Set`) and the time-synchronised `IsAtHome` condition evaluates to False, the logic defaults to the active Patrol sequence. This cross-system integration ensures that the local spatial behaviour of the BT adheres to the global temporal schedule.

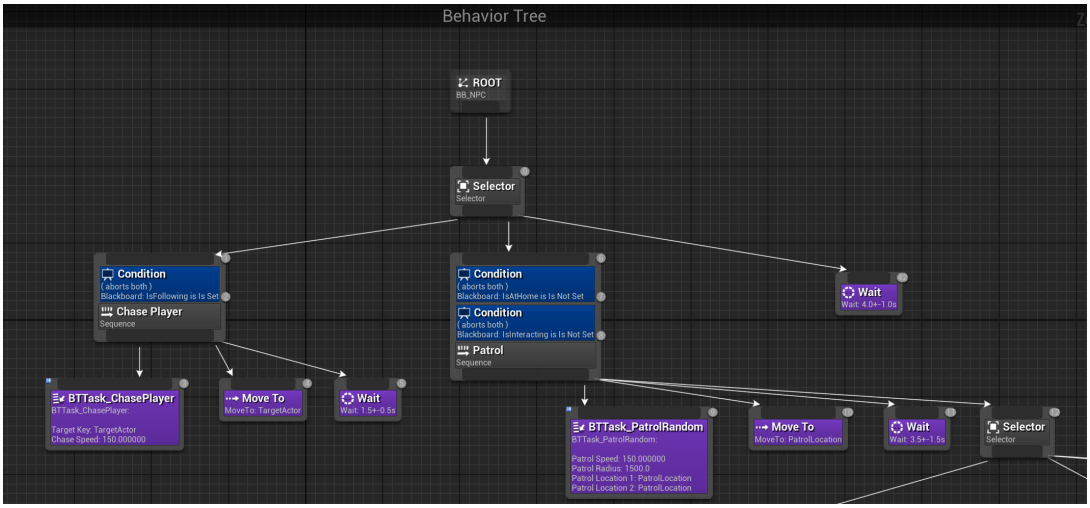


Figure 17: Top-level hierarchy of the NPC Behaviour Tree, utilising Blackboard Decorator conditions to pivot between chasing, patrolling, and idling states.

A significant challenge in AI design is the prevention of mechanical, predictable looping, which rapidly diminishes player immersion [7]. To synthesize organic behaviour during downtime, a stochastic animation matrix was integrated into the BT. Instead of a single idle animation, a `Selector` node branches into numerous `Play Animation` tasks, each gated by a `BTD_RandomChance` decorator.

When the NPC halts, the tree evaluates these decorators. As shown in Figure 18, chances range from 10% to 33%. These distribution weights were empirically tuned through iterative playtesting to strike a behavioural balance, ensuring NPCs do not appear unnaturally hyperactive nor remain dormant for immersion-breaking durations. Depending on the evaluation, the NPC may dynamically execute routines such as stretching, looking around nervously, performing push-ups, or picking fruit. This probability-based distribution ensures that the village ecosystem feels perpetually alive and unpredictable.

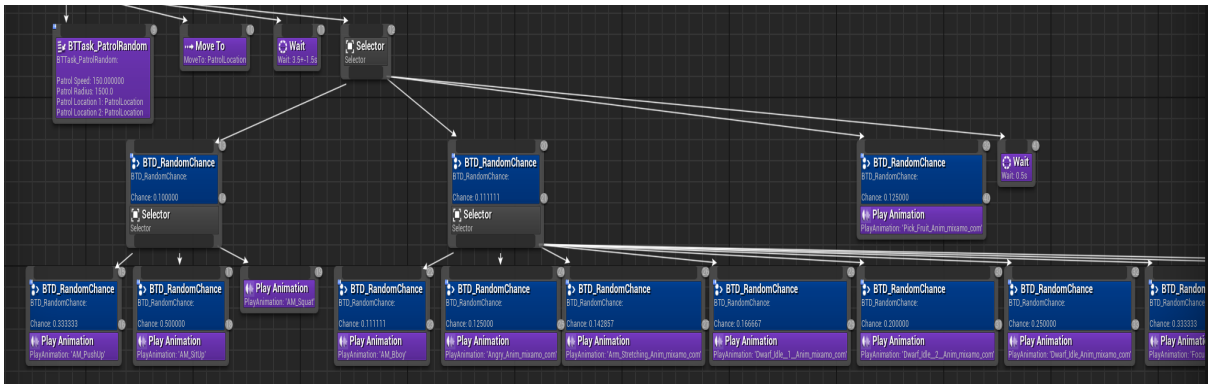


Figure 18: Stochastic behavioural logic implemented via Random Chance decorators, injecting procedural variety into NPC idle states.

4.3.3 Animation Pipeline and Embedded Auditory Feedback

The visual realization of these AI decisions relies on a 1D Blend Space (`BS_MF_Unarmed_WalkRun`). Sourced primarily from Mixamo and retargeted to the Unreal Engine skeleton [27, 28], the Blend Space interpolates between discrete animation sequences (`Idle`, `Walk`, `Run`) based on the pawn's current `Speed` scalar ($0.0 \sim 600.0$), where the default walking speed of 300.0 occupies the midpoint of this range and the upper values accommodate sprint and Special Item-modified states [29].

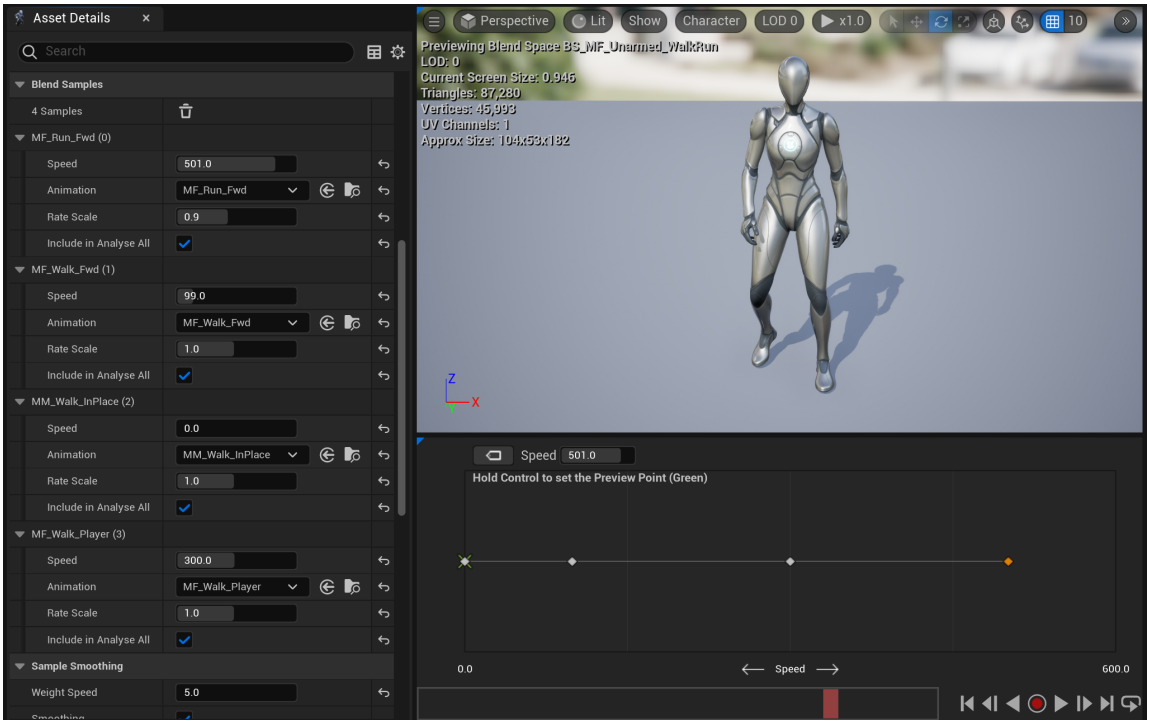


Figure 19: 1D Blend Space mapping velocity scalars to animation sequences, featuring a dedicated player walk cycle for selective audio binding.

A custom engineering solution was implemented within this animation pipeline to handle auditory feedback. Rather than a scope limitation, the conscious omission of a global audio manager was a deliberate design decision to prevent audio channel saturation from multiple NPCs. To achieve this, the walking animation was duplicated and bifurcated into an NPC version (`MF_Walk_Fwd`) and a Player version (`MF_Walk_Player`).

For the player's specific sequence, `Animation Notifies` (AnimNotifies) were embedded directly into the timeline at the frames where the character's heel strikes the ground. These notifies trigger footstep Sound Cues. By binding the auditory feedback directly to the kinematic visual data of the player, rather than relying on global timers, the system

ensures precise audio-visual synchronization. This separation ensures that NPCs do not flood the audio channels with footsteps, whilst the player receives responsive, tactile sensory feedback, supporting the SCI (Sensory, Challenge, Imagination) immersion model [12].

4.4 Context-Aware Interaction System

The architectural foundations established in the preceding sections—namely, the global time manager and the player's physiological state mechanics—provide the necessary data inputs for a responsive game world. However, the qualitative success of the prototype, as measured against the immersion failure modes identified in Section 1.2, relies on how NPCs interpret and react to this data. Section 4.4 details the engineering of the Context-Aware Interaction System, a multi-layered architecture encompassing precise spatial targeting, state-driven dialogue prioritization, and probabilistic relationship mechanics mapped to distinct player inputs ('E', 'R', and 'F' keys).

4.4.1 Spatial Proximity and Intent Disambiguation

Before a meaningful interaction can occur, the system must deterministically resolve which entity the player intends to engage with. To establish a robust baseline for proximity detection, a `Sphere Collision Component` with a radius of 300.0 units was bound to the NPC base class. When a player breaches this threshold, an event trigger dynamically renders an in-game `Widget Component` displaying an interaction prompt above the NPC's head (Figure 20), providing immediate diegetic feedback [16].

Figure 20: In-game contextual UI prompt triggered by overlapping the NPC's collision bounds.

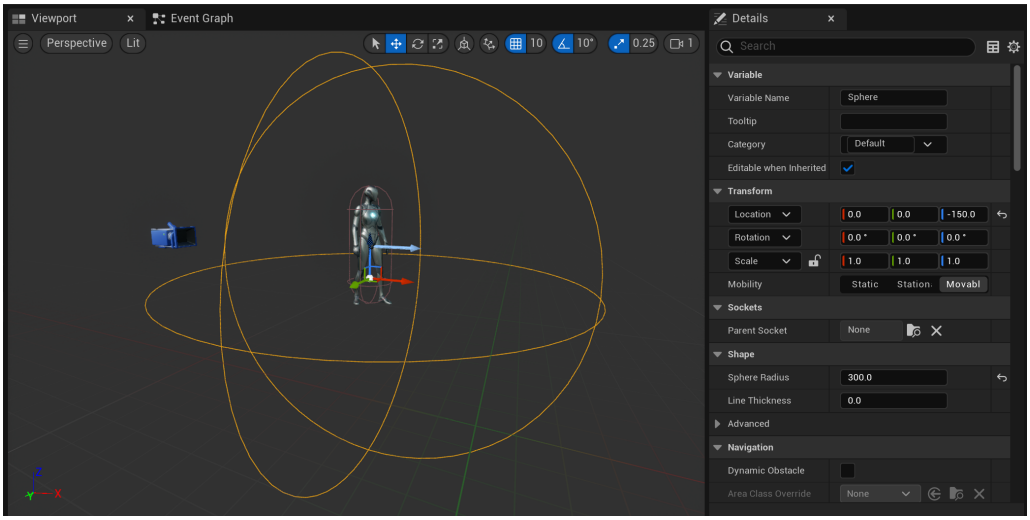


Figure 21: NPC interaction boundary, defined by a 300.0-unit Sphere Collision component.

However, relying solely on radial overlaps presents a critical edge case in dense environments: "interaction hijacking," where the system inadvertently selects a peripheral NPC rather than the one the player is directly facing. To resolve the ambiguity of multiple overlapping boundaries, a custom "Look-At" disambiguation algorithm was engineered using vector mathematics.

When the player triggers the primary interaction input ('E' Key), the system retrieves an array of all actors currently overlapping the 300.0-unit sphere. A `For Each Loop` iterates through this array, performing a Dot Product (scalar product) operation between the player camera's forward-facing vector ($\vec{V}_{fwd}$) and the normalized directional vector from the player to the evaluated NPC ($\vec{V}_{npc}$). The evaluation is formalized as:

$$Score = \vec{V}_{fwd} \cdot \vec{V}_{npc} = |\vec{V}_{fwd}||\vec{V}_{npc}| \cos(\theta)$$

Because both vectors are normalized, the resulting scalar score ranges from -1.0 to 1.0. A score approaching 1.0 indicates that the NPC is located closest to the centre of the player's Field of View (FOV). As illustrated in Figure 22, the algorithm continuously evaluates these dot products against a cached `Best Score` variable, isolating the maximum value. The winning actor is assigned as the `Best Target`, ensuring reliable intent resolution before dispatching the `Send Interaction Signal` via Blueprint Interfaces. The empirical resilience of this disambiguation algorithm against interaction hijacking within dense, overlapping NPC clusters is validated through the functional testing scenarios detailed in Chapter 5.

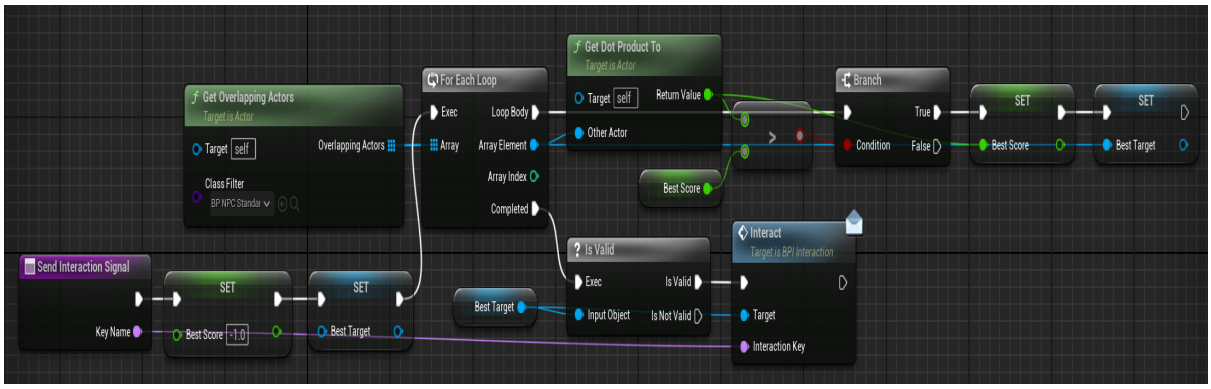


Figure 22: Target disambiguation algorithm utilizing vector Dot Products to identify the highest alignment score within overlapping NPC boundaries.

Upon receiving the interaction signal, the selected NPC must physically acknowledge the player. Mechanical, instantaneous rotation undermines the illusion of a living agent. Therefore, a procedural kinematics sequence was implemented using the `Find Look at Rotation` node. To prevent the NPC's mesh from contorting unnaturally when the player is positioned at a different elevation (e.g., stairs or uneven terrain), the Pitch and Roll axes are explicitly clamped to 0.0. Only the Z-axis (Yaw) is dynamically updated. This

constrained rotator is processed through an `RInterp To` node with an interpolation speed of 5.0, resulting in a smooth, human-like pivot towards the player [27], as shown in Figure 23.

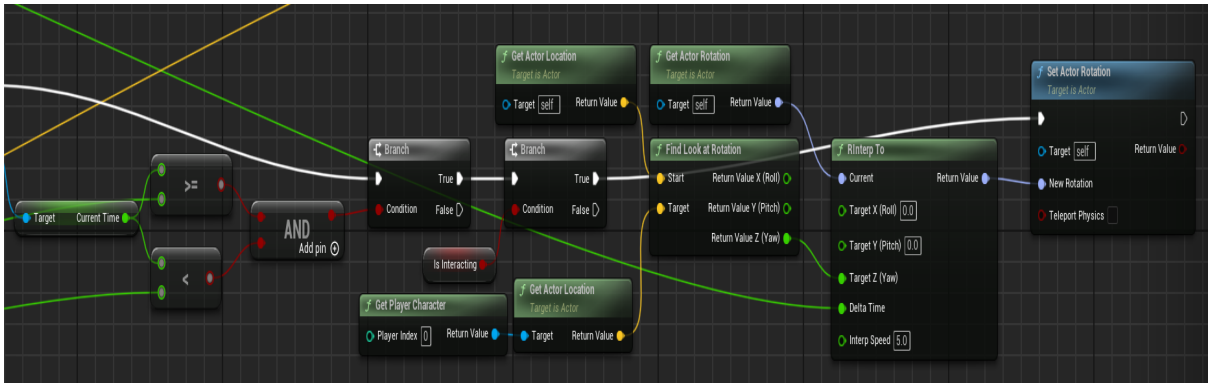


Figure 23: Procedural kinematics logic clamping Roll and Pitch to 0.0, utilizing `RInterp To` on the Yaw axis for smooth rotational alignment.

4.4.2 State-Driven Dialogue Priority Engine

The fundamental failure mode of traditional dialogue trees is their context-blindness; NPCs will typically offer a cheerful greeting regardless of whether the player is exhausted or impoverished [12]. To address this cognitive dissonance, a State-Driven Dialogue Priority Engine was developed.

When the player initiates a secondary deep interaction ('R' Key), the system preemptively evaluates the player's physiological and economic data against predefined thresholds. As documented in Figure 24, the engine first executes a `Branch` node checking if `CurrentStamina` ≤ 30.0. This specific threshold represents an intentional UX design choice. As established in Section 4.2.1, the player suffers severe movement penalties when stamina drops below 20.0, with passive decay removing 10.0 stamina per in-game hour. Therefore, the 30.0 threshold triggers precisely one in-game hour before critical exhaustion. If True, the NPC fetches the `Stamina` array from the `My Dialogue Map` dictionary, delivering a contextual warning regarding the player's visible fatigue, effectively acting as an immersive, diegetic early-warning system.

If the stamina check returns False, the engine subsequently evaluates if `Money` $== 0.0$. This threshold is equally significant because the lowest-tier stamina restoration item in the prototype's economy costs exactly 1 currency unit. Absolute zero equates to an impending survival lock. In this scenario, the engine fetches the `Money` array, prompting the NPC

to acknowledge the player's destitution. By routing conversational flow through these Boolean gates, the system dynamically surfaces vital survival metrics through natural conversation, reinforcing narrative believability [20].

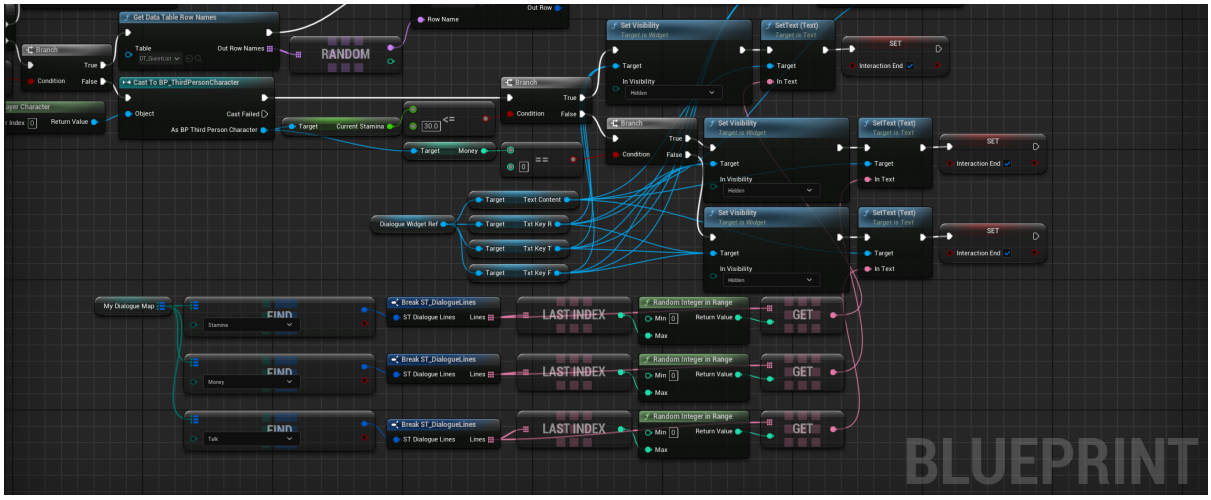


Figure 24: State-Driven Priority Engine: Boolean gates check for pre-exhaustion (Stamina ≤ 30.0) and economic locks (Money $== 0.0$) to override standard dialogue.

4.4.3 Probabilistic Quests and Companion System

When the Priority Engine determines that the player is in a stable state, the system opens up the possibility for deeper narrative engagement. To maintain the unpredictability of the ecosystem, a stochastic quest generation system was integrated. As shown in Figure 25, when a standard conversation is initiated, a `Random Float in Range` ($0.0 \sim 1.0$) is generated and compared against a ≤ 0.1 condition. This establishes a 10% probability of triggering a dynamic quest from the `DT_QuestList` Data Table, leaving the remaining 90% to populate ambient, personality-driven world-building dialogue [31].

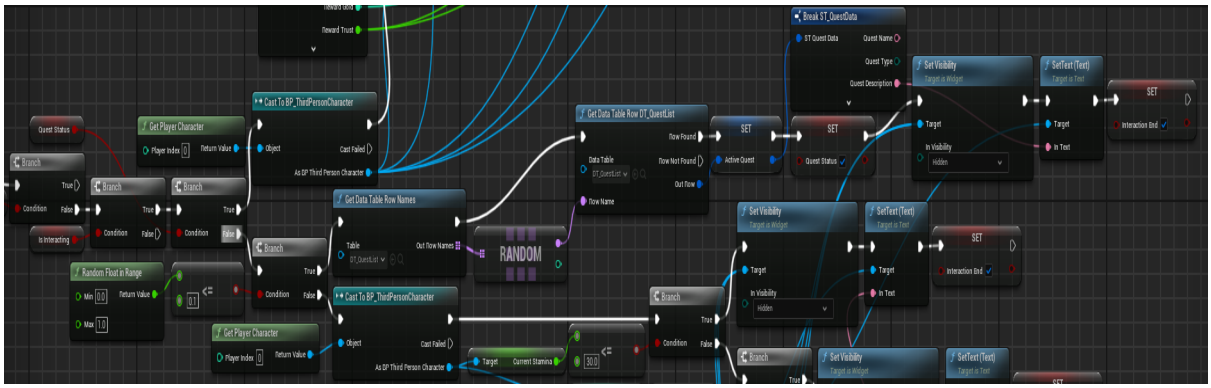


Figure 25: Stochastic execution path for quest generation, utilizing a 10% probability threshold to balance ambient dialogue and active tasks.

Successfully resolving a quest applies an addition to the player's `Money` and the NPC's `Trust Level`. This accumulation of `Trust Level` serves as the primary relationship progression mechanic [19]. The functional culmination of this trust system is the "Companion Override" mechanic, bound to the 'F' Key interaction.

When the player requests an NPC to follow them, a key `Branch` node evaluates if the current `Trust Level` is greater than or equal to the NPC's `Required_Trust_Follow` threshold (e.g., 80.0 for 'Charles', 40.0 for 'George', as fetched from the master Data Table). These disparate thresholds are not arbitrary; they are deliberate game design choices that encode the NPCs' narrative personalities. For instance, Charles's 'Busy' and introverted archetype necessitates a significantly higher sustained social investment to earn companionship compared to George's naturally 'Talkative' and extroverted disposition. By embedding psychological traits into data parameters, the system achieves narrative depth without requiring bespoke logic for each character. If the trust is insufficient, the system fetches a rejection dialogue string. However, if the threshold is satisfied, the system updates the Blackboard memory key `IsFollowing` to True (Figure 26). This single Boolean toggle overrides the primary Behaviour Tree Selector (Section 4.3.2), causing the NPC to abandon its time-driven patrol schedule and dynamically pathfind to trail the player character, thereby directly addressing the static relationship failure mode.

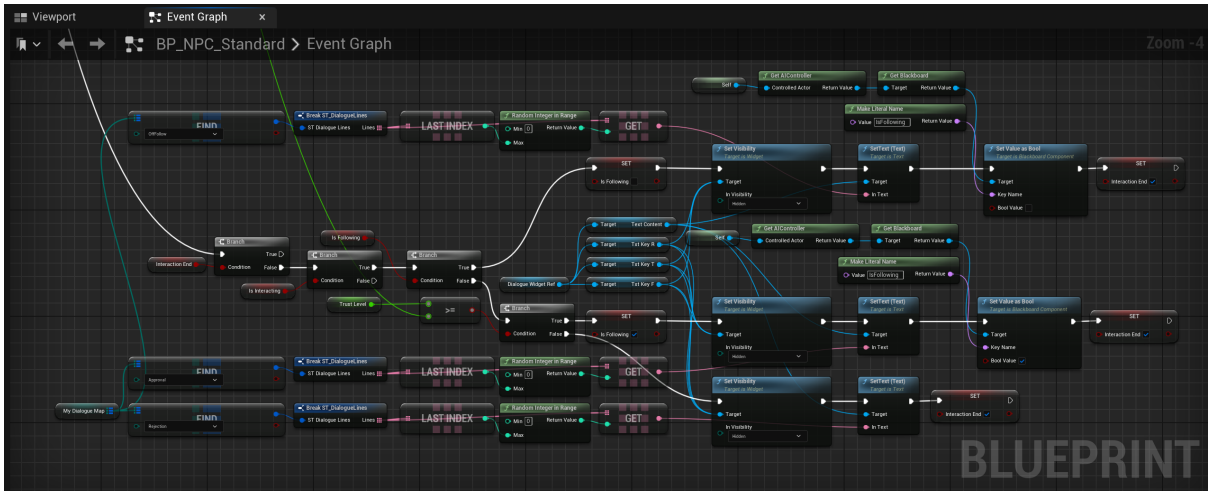


Figure 26: Trust progression logic governing the companion override. The branch evaluates accumulated trust against Data Table thresholds to unlock the Follow behaviour.

4.5 Economy, Trading, and Inventory Management

The final architectural pillar of the prototyped NPC ecosystem is the integration of a dynamic, trust-gated economy. In traditional role-playing games, merchants often function as static vending machines, entirely divorced from the game's narrative or relationship mechanics [23]. To counter the static relationship failure mode identified in Section 1.2, this project engineered an integrated socio-economic system. Section 4.5 details the technical implementation of the memory-efficient procedural foraging system, the event-driven contextual trading interface, and the probabilistic algorithms governing trust-gated commerce.

4.5.1 Data-Driven Inventory Architecture and Procedural Foraging

To manage the player's economic assets and consumable resources without degrading runtime performance, a struct-based array inventory architecture was engineered. In expansive open-world environments, instantiating and maintaining hundreds of individual 3D item actors in memory (via `SpawnActor`) can lead to Garbage Collection (GC) spikes and framerate degradation [24]. To circumvent this, the system treats physical items purely as temporary interaction triggers.

When a player physically overlaps with a gatherable item's Sphere Collision, the interaction logic is triggered. Upon execution, the system does not store the actor itself; instead, it parses the item's data into a predefined struct and destroys the physical actor. The system then iterates through the player's `Inventory Array`, performing an $O(N)$ linear search to check if an item with an identical string identifier already exists. If the Boolean condition evaluates to True, the system accesses that array index and increments its integer count. If False, it appends a new struct element to the array. This 'search-and-stack' algorithm reduces memory overhead, prevents array bloat, and simplifies data serialization for save states.

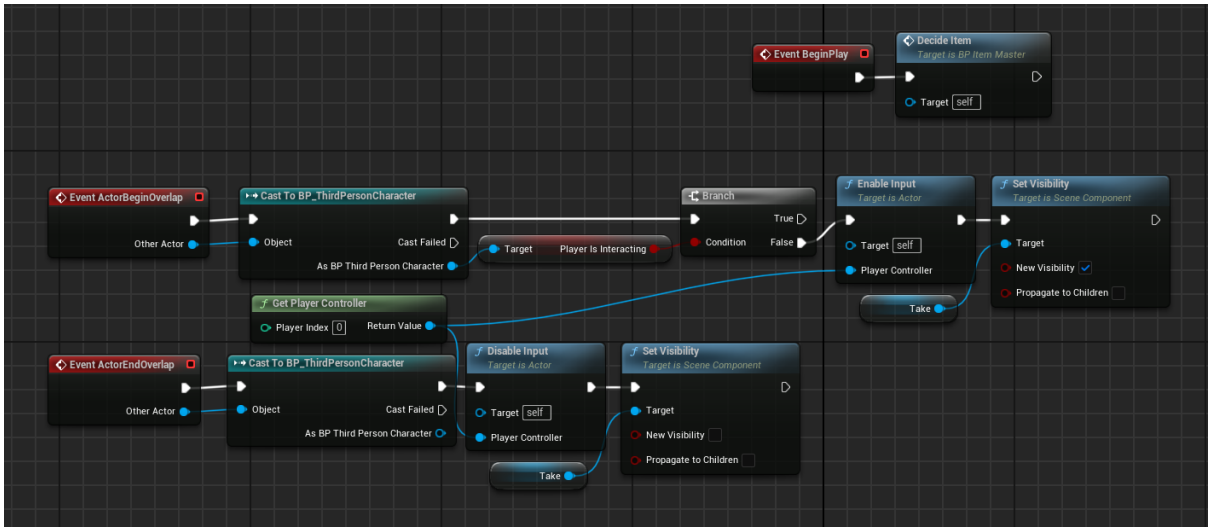


Figure 27: Interaction logic for gatherable items. Physical actors are destroyed upon overlap and their data is transferred to the player's inventory struct.

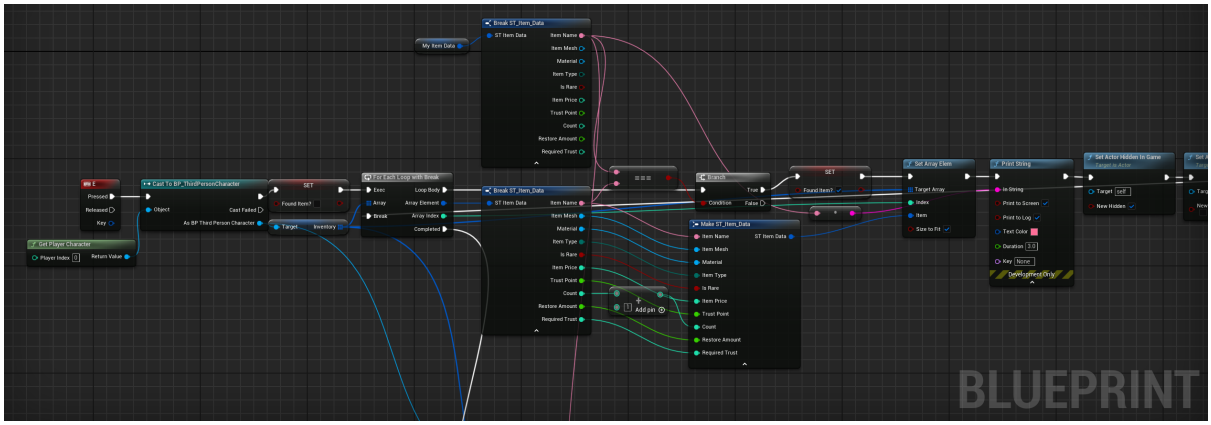


Figure 28: Inventory array manipulation utilizing a `For Each Loop`. The $O(N)$ algorithm increments counts for existing identifiers, optimizing memory usage.

Furthermore, to populate the world efficiently, a 'Master Item' procedural generation blueprint was developed. Instead of creating dozens of unique, hard-coded item classes, a single generic actor dynamically alters its visual material properties and nomenclature based on localized parameters. A notable feature of this system is the stochastic generation of 'Rare' items, which introduces economic unpredictability. When an item instantiates, a `Random Float in Range` $(0.0 \sim 1.0)$ is evaluated against a ≤ 0.10 threshold. This 10% probability procedurally modifies the item: it shifts the material parameter to an emissive golden hue and prefixes the string identifier with "Rare".

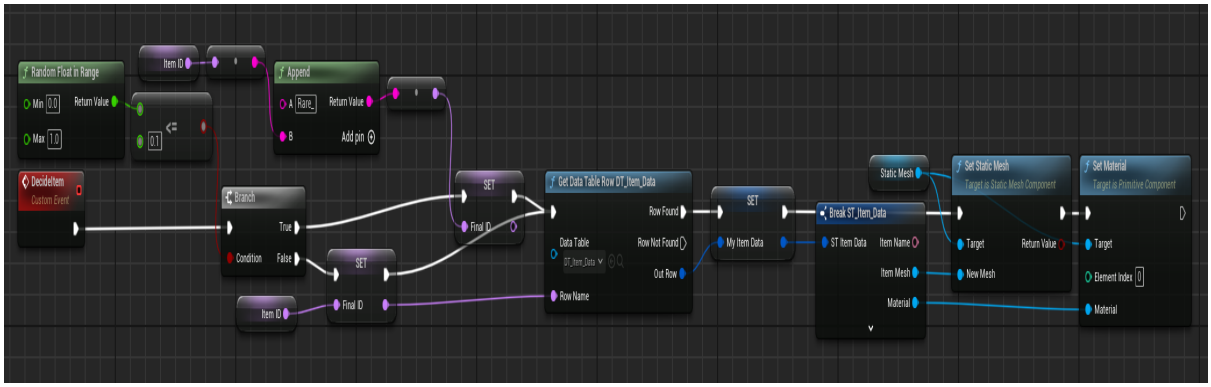


Figure 29: Procedural item generation with 10% stochastic probability ($Float \leq 0.1$) to instantiate 'Rare' variants from the Master Item blueprint.

4.5.2 Event-Driven Trading Interface and Contextual UX

To facilitate economic exchange, a User Interface (UI) was constructed utilizing a modified Model-View-Controller (MVC) paradigm. The system segregates interaction intents through specific input bindings: the 'I' key invokes a standalone personal inventory for item consumption, whilst the 'T' key—accessible exclusively during an active spatial overlap with an NPC—initializes the comprehensive trading menu.

When the 'T' key is executed, the UI renders a dual-panel grid, displaying the NPC's specific stock on the left and the player's inventory on the right. To optimize rendering performance, this UI is event-driven. Rather than relying on computationally expensive `Event Tick` functions to update the display, the UI text blocks are bound to dedicated variables. Any transaction instantly recalculates the underlying data and pushes a direct update to the `Money` and `Trust Level` widgets, ensuring visual accuracy with zero frame-latency.

Figure 30: The standalone personal inventory UI, triggered via the 'I' key, isolated from commercial functionality.

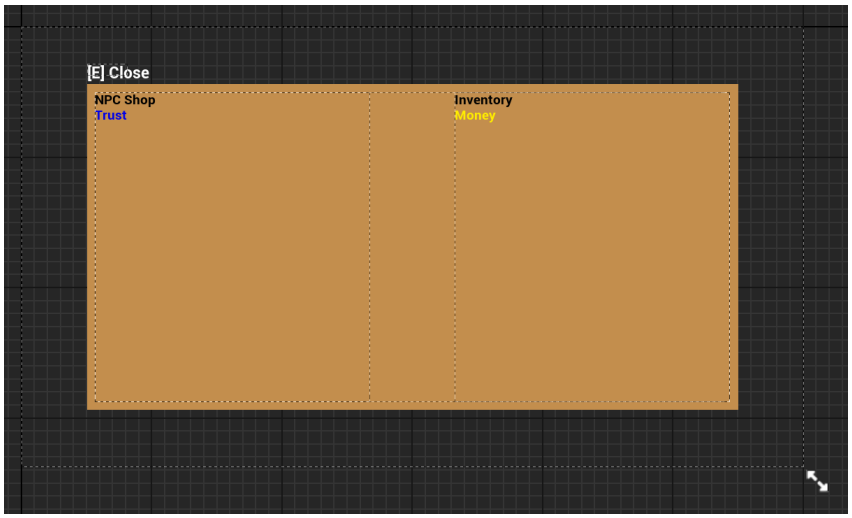


Figure 31: The Trading UI, displaying side-by-side inventory grids alongside event-driven variables for Money and NPC Trust Level.

A major challenge in UI engineering is preventing logical errors and invalid user transactions, such as a player attempting to consume an item directly from the NPC's stock before purchasing it. To resolve this, a Contextual Button Visibility system was engineered. The underlying UI blueprint continuously evaluates the active interaction mode and verifies the ownership array of the selected item struct. By manipulating the `ESlateVisibility` enumerator (toggling between `Visible`, `Collapsed`, and `Hidden`), the system programmatically restricts the exposure of the 'Use', 'Buy', and 'Sell' buttons to match the context

of the user's selection. This state-sensitive UX design ensures a frictionless interaction flow that prevents transactional errors [16].

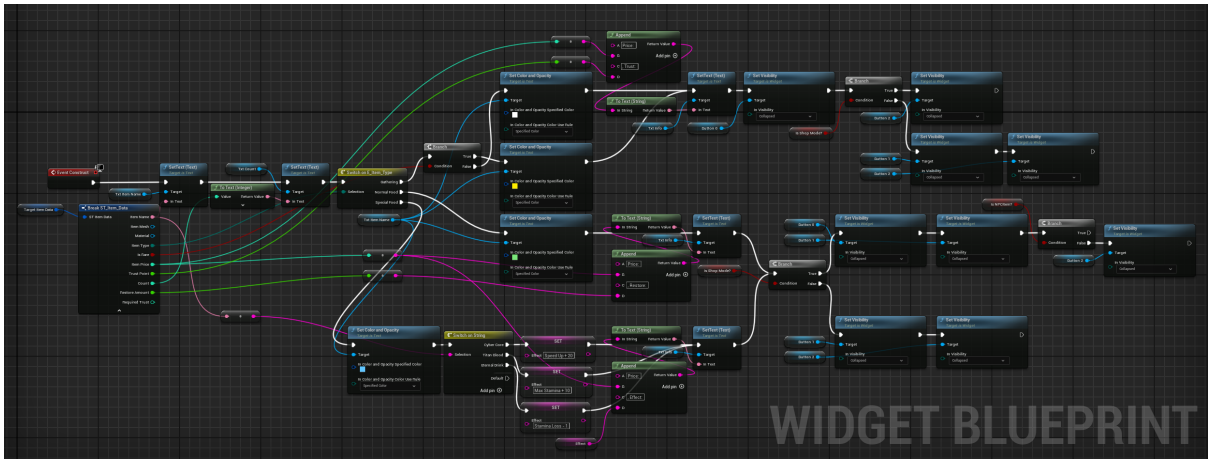


Figure 32: Contextual UX logic manipulating `ESlateVisibility` to toggle button exposure (Use, Buy, Sell) based on ownership state.

4.5.3 Socio-Economic Coupling: Addressing Static Relationships

A key design contribution within the prototype's economic framework is the mechanical coupling of financial transactions to the social relationship system. In conventional RPG architectures, economy and narrative trust are isolated systems. In this prototype, they are mutually dependent variables designed to directly address the failure mode of static, unevolving NPC interactions [19].

This coupling is highlighted through the handling of the procedurally generated 'Rare' items and the quest resolution logic. While selling a standard food item executes a simple arithmetic addition to the player's `Money` pool, selling a 'Rare' item triggers a specialized parallel execution path. The logic utilizes a `Contains` string parsing node. If it identifies the "Rare" string prefix within the item's name, it bypasses the standard loop to simultaneously deposit the currency and apply a positive integer scalar to the specific NPC's local `Trust Level`.

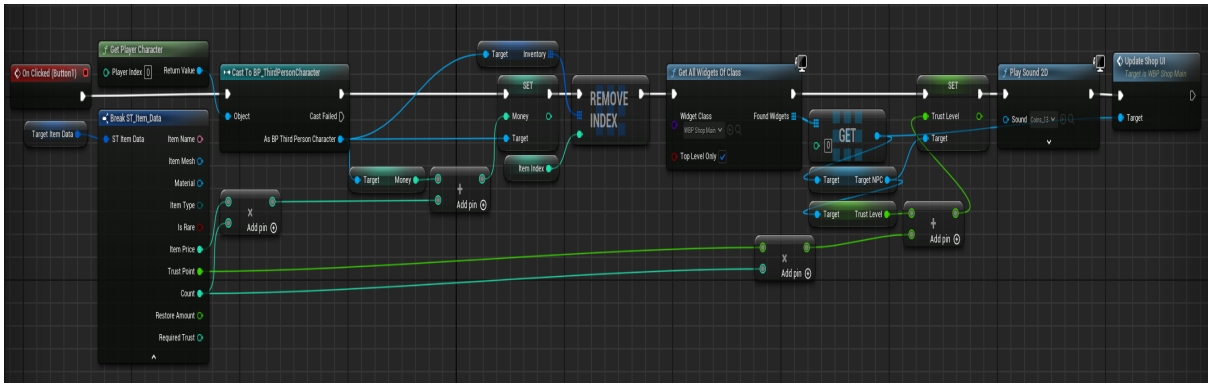


Figure 33: Socio-economic coupling via string parsing. Selling 'Rare'-prefixed items simultaneously increments player currency and NPC Trust.

This dual-reward mechanic is also mirrored in the questing system. When the player resolves a gathering quest, the system evaluates the rarity of the submitted items. Turning in rare variants triggers a distinct execution branch that amplifies both the financial payout and the relational trust gain. This integration transforms open-world foraging into a viable alternative pathway for relationship progression, accommodating diverse player motivations and demonstrating that procedural content generation can meaningfully impact narrative systems [31].

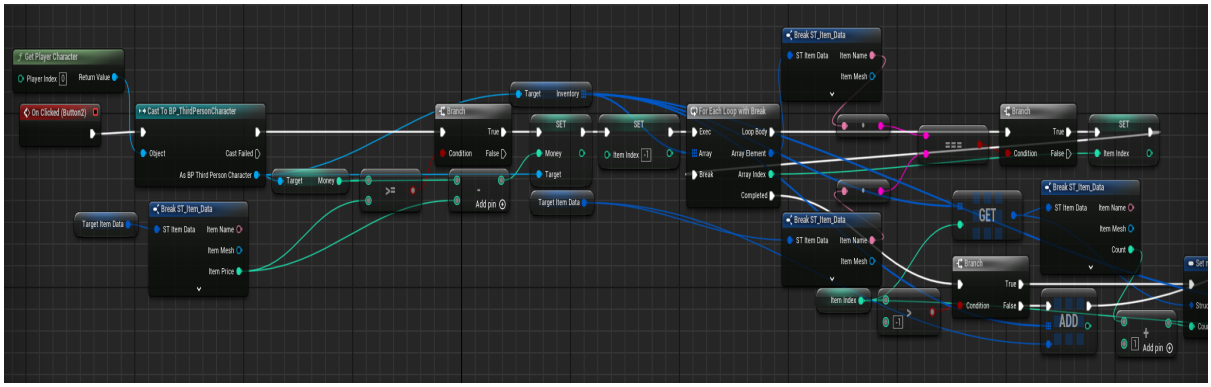


Figure 34: Quest resolution logic with differentiated Trust and Money rewards based on the procedural rarity of gathered items.

4.5.4 Trust-Gated Commerce and Special Modifiers

The culmination of this interconnected ecosystem is the implementation of "Trust-Gated Commerce". To address the issue of static NPC utility and provide long-term gameplay goals, the highest tier of the village economy is restricted by a social barrier. While basic consumables (e.g., standard stamina-restoring food) are universally purchasable to maintain the survival loop, 'Special Items' are guarded by the NPCs.

These Special Items are not merely cosmetic; they impart permanent mechanical modifiers to the player's physical capabilities. Through custom execution events, consuming these items interacts directly with the `CharacterMovementComponent` and the survival timer established in Section 4.2. The three implemented modifiers include: global movement speed amplification, maximum stamina pool expansion, and stamina decay rate mitigation.

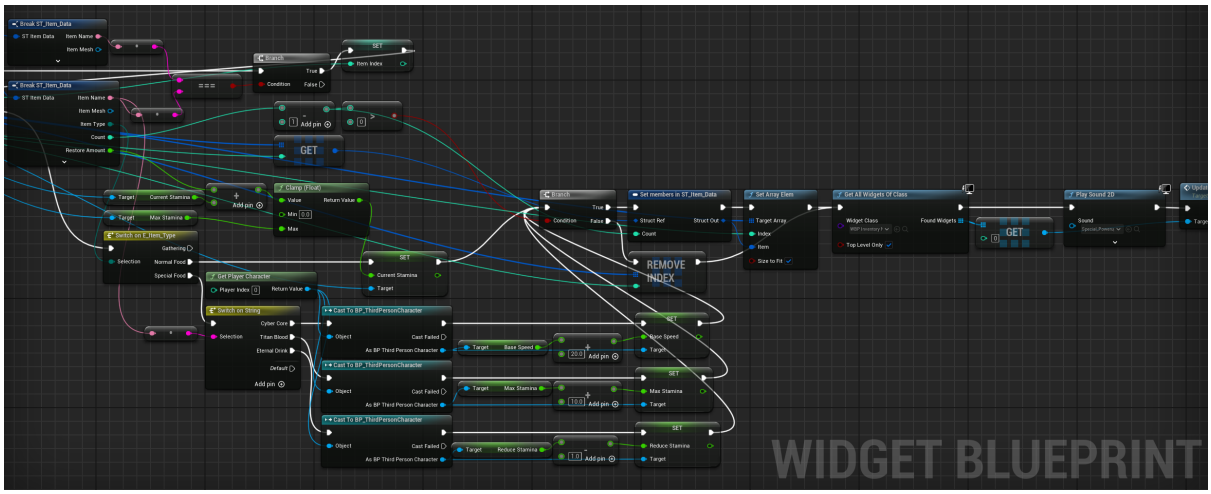


Figure 35: Blueprint logic defining permanent modifiers applied to the player's movement and survival components upon consuming Special Items.

When the player attempts to purchase a Special Item, a key `Branch` node intercepts the execution flow. It reads the NPC's active `Trust Level` and evaluates it against a specific parametric threshold (currently configured to ≥ 100.0 for universal testing, but exposed as a modular variable for future balancing). If the condition evaluates to False, the Special Items are conditionally excluded from the NPC's visible trading inventory entirely, communicating to the player that financial wealth alone cannot bypass social bonds.

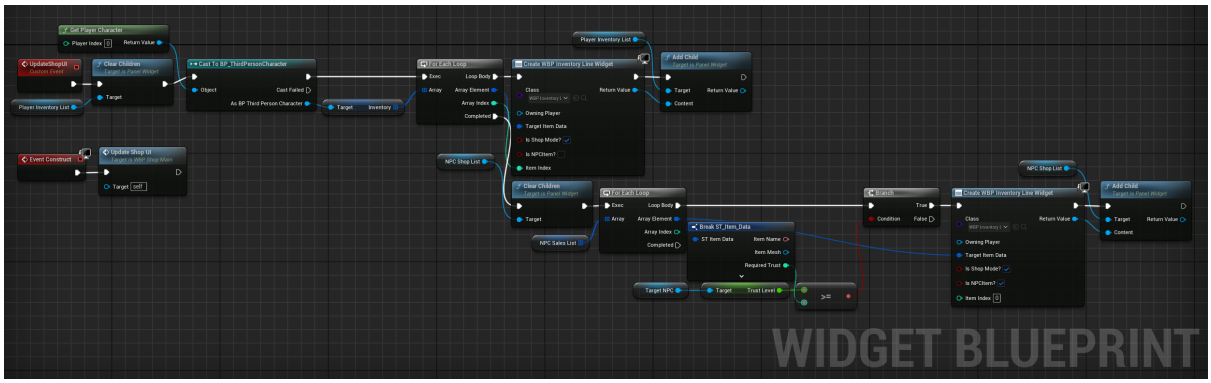
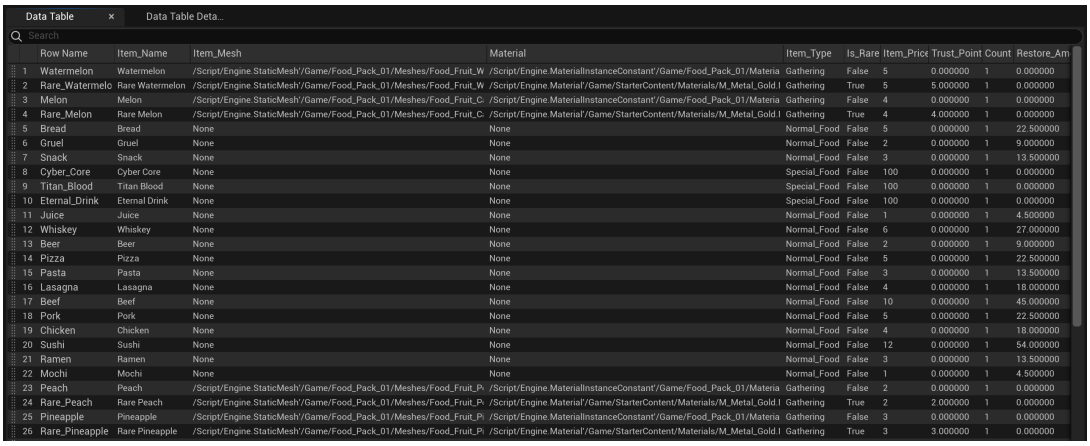


Figure 36: Trust-gated transaction logic evaluating whether the NPC's Trust Level ≥ 100.0 before permitting the transfer of Special Items.

Additionally, the data structures governing NPC inventories are asymmetrically distributed. Different NPCs sell different standard foods at varying price points, and certain NPCs do not stock Special Items at all. This deliberate, data-driven design encourages spatial exploration; players cannot exploit a single centralized vendor. Instead, they are mechanically incentivized to navigate the village, map individual NPC economies, and cultivate sustained relationships with multiple characters to acquire the full suite of upgrades. By positioning the most powerful gameplay mechanics behind an earned social threshold, the system demonstrates that immersive NPC relationships can serve as foundational pillars of a compelling gameplay loop [22, 26].



Data Table × Data Table Deta...

Search

	Row Name	Item_Name	Item_Mesh	Material	Item_Type	Is_Rare	Item_Price	Trust_Point	Count	Restore_Am
1	Watermelon	Watermelon	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_W	/Script/Engine.MaterialInstanceConstant'/Game/Food_Pack_01/Materia	Gathering	False	5	0.000000	1	0.000000
2	Rare_Watermelo	Rare Watermelon	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_W	/Script/Engine.Material'/Game/StarterContent/Materials/M_Metal_Gold.I	Gathering	True	5	5.000000	1	0.000000
3	Melon	Melon	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_C	/Script/Engine.MaterialInstanceConstant'/Game/Food_Pack_01/Materia	Gathering	False	4	0.000000	1	0.000000
4	Rare_Melon	Rare Melon	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_C	/Script/Engine.Material'/Game/StarterContent/Materials/M_Metal_Gold.I	Gathering	True	4	4.000000	1	0.000000
5	Bread	Bread	None	None	Normal_Food	False	5	0.000000	1	22.500000
6	Gruel	Gruel	None	None	Normal_Food	False	2	0.000000	1	9.000000
7	Snack	Snack	None	None	Normal_Food	False	3	0.000000	1	13.500000
8	Cyber_Core	Cyber Core	None	None	Special_Food	False	100	0.000000	1	0.000000
9	Titan_Blood	Titan Blood	None	None	Special_Food	False	100	0.000000	1	0.000000
10	Eternal_Drink	Eternal Drink	None	None	Special_Food	False	100	0.000000	1	0.000000
11	Juice	Juice	None	None	Normal_Food	False	1	0.000000	1	4.500000
12	Whiskey	Whiskey	None	None	Normal_Food	False	6	0.000000	1	27.000000
13	Beer	Beer	None	None	Normal_Food	False	2	0.000000	1	9.000000
14	Pizza	Pizza	None	None	Normal_Food	False	5	0.000000	1	22.500000
15	Pasta	Pasta	None	None	Normal_Food	False	3	0.000000	1	13.500000
16	Lasagna	Lasagna	None	None	Normal_Food	False	4	0.000000	1	18.000000
17	Beef	Beef	None	None	Normal_Food	False	10	0.000000	1	45.000000
18	Pork	Pork	None	None	Normal_Food	False	5	0.000000	1	22.500000
19	Chicken	Chicken	None	None	Normal_Food	False	4	0.000000	1	18.000000
20	Sushi	Sushi	None	None	Normal_Food	False	12	0.000000	1	54.000000
21	Ramen	Ramen	None	None	Normal_Food	False	3	0.000000	1	13.500000
22	Mochi	Mochi	None	None	Normal_Food	False	1	0.000000	1	4.500000
23	Peach	Peach	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_P	/Script/Engine.MaterialInstanceConstant'/Game/Food_Pack_01/Materia	Gathering	False	2	0.000000	1	0.000000
24	Rare_Peach	Rare Peach	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_P	/Script/Engine.Material'/Game/StarterContent/Materials/M_Metal_Gold.I	Gathering	True	2	2.000000	1	0.000000
25	Pineapple	Pineapple	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_P	/Script/Engine.MaterialInstanceConstant'/Game/Food_Pack_01/Materia	Gathering	False	3	0.000000	1	0.000000
26	Rare_Pineapple	Rare Pineapple	/Script/Engine.StaticMesh'/Game/Food_Pack_01/Meshes/Food_Fruit_P	/Script/Engine.Material'/Game/StarterContent/Materials/M_Metal_Gold.I	Gathering	True	3	3.000000	1	0.000000

Figure 37: Asymmetrical inventory configuration via Data Tables, with unique arrays assigned to individual NPCs to encourage multi-character relationship building.

5 Evaluation and Testing

The evaluation of the prototype follows the structured tripartite methodology established in Section 3.5. Jennett et al. [13] argue that meaningful evaluation of immersive systems requires both quantitative measurement and structured qualitative assessment; accordingly, this chapter systematically verifies the functional correctness of individual subsystems (Section 5.1), assesses systemic coherence under combined edge-case conditions (Section 5.2), empirically validates the probabilistic systems through repeated trials (Section 5.3), and concludes with a structured mapping of the prototype's outcomes against the three failure modes identified in Section 1.2 (Section 5.4).

5.1 Functional Unit Testing

Before evaluating the prototype's holistic player experience, it is necessary to verify that each individual subsystem operates correctly in isolation. Millington and Funge [7] emphasise that modular verification of individual AI components is a prerequisite for reliable integration in complex game systems. A series of controlled unit-level tests was designed and executed, targeting the core mechanics established in Chapter 4. Table 4 summarises the results.

Subsystem	Test Condition	Expected Outcome	Result
Time System (§4.1.1)	Allow `CurrentTime` to exceed 1440.0	Modulo resets value to 0.0; no overflow	**Pass**
Stamina Decay (§4.2.1)	Observe stamina at 60-second intervals	Decrement by `ReduceStamina`; Clamp to [0, 100]	**Pass**
Movement Penalty (§4.2.1)	Stamina ≥ 20; then Stamina < 20	Speed = 300.0; then Speed = 200.0	**Pass**
Dialogue Priority (§4.4.2)	Interact with Stamina ≤ 30	NPC delivers exhaustion warning first	**Pass**
Dialogue Priority (§4.4.2)	Interact with Money == 0	NPC delivers poverty-specific dialogue	**Pass**
Inventory Stacking (§4.5.1)	Collect duplicate items	Count increments; no duplicate array entry	**Pass**
Trust — Follow (§4.4.3)	Request Follow with Trust $<$ threshold	NPC rejects with dialogue string	**Pass**
Trust — Follow (§4.4.3)	Request Follow with Trust $\geq$ threshold	NPC enters Follow state via BT override	**Pass**
Trust — Commerce (§4.5.4)	Purchase Special Item with Trust < 100	Special Items hidden from NPC inventory	**Pass**
Trust — Commerce (§4.5.4)	Purchase Special Item with Trust ≥ 100	Special Items visible and purchasable	**Pass**
NPC Lifecycle (§4.1.3)	Observe NPC at Start/End Time boundaries	Spawn at Start; navigate home and despawn at End	**Pass**

Table 4: Functional unit test results for individual subsystems. All eleven tests returned expected outcomes under controlled conditions.

5.1.1 Movement Speed Verification

A representative example of the functional testing process is the verification of the stamina-driven movement penalty. As established in Section 4.2.1, the system enforces a binary speed state consistent with the legibility principles advocated by Salen and Zimmerman [22]: a base `MaxWalkSpeed` of 300.0 when `CurrentStamina` ≥ 20.0, and a penalised speed of 200.0 when stamina drops below this threshold. Figure 38 confirms this behaviour

through runtime variable inspection within the Unreal Engine 5 debugging environment [1].

Figure 38: Movement speed verification: healthy state (Stamina ≥ 20, Speed $= 300$) on the left versus exhausted state (Stamina < 20, Speed $= 200$) on the right.

5.1.2 State-Driven Dialogue Verification

The dialogue priority engine was tested by manipulating the player's stamina and currency values to trigger each conditional branch independently. Fagerholt and Lorentzon [16] argue that surfacing critical state information at the point of interaction reduces cognitive overhead; this test verifies that the engine implements this principle correctly. Figure 39 demonstrates the system's contextual responsiveness: when stamina falls below 30, the NPC overrides its standard greeting with an exhaustion-specific warning; when money reaches zero, the NPC acknowledges the player's economic state. In both cases, the priority logic correctly intercepts the default dialogue path, confirming the state-driven architecture operates as designed.

Figure 39: Dialogue priority verification: exhaustion warning triggered at Stamina ≤ 30 (left) and poverty-specific dialogue triggered at Money $== 0$ (right).

5.1.3 Trust-Gated Follow Verification

The trust-based companion system was validated by testing the Follow mechanic at both sides of the required threshold. Ryan, Rigby, and Przybylski [19] identify the sense of relatedness as a fundamental driver of player engagement; the trust-gated Follow mechanic operationalises this by requiring sustained social investment before granting companionship. As shown in Figure 40, when the player's accumulated trust with an NPC falls below the required value, the system correctly rejects the request with a character-appropriate dialogue response [20]. Upon reaching the threshold, the same interaction succeeds, and the NPC transitions to the Follow state via the Behaviour Tree override mechanism [5] documented in Section 4.4.3.

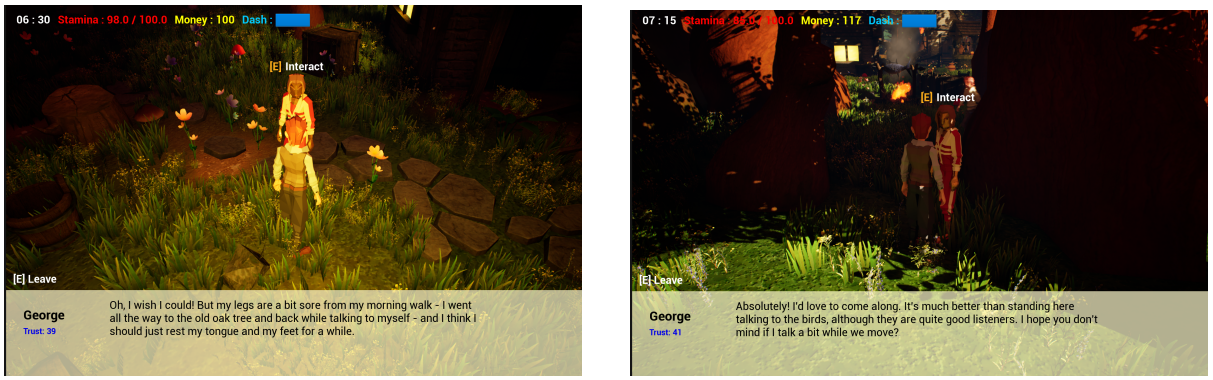


Figure 40: Follow mechanic verification: request rejected at insufficient trust (left) versus accepted upon reaching the threshold (right).

5.1.4 Trust-Gated Commerce Verification

The Special Item commerce gate was tested independently from the Follow mechanic. Adams [23] notes that economic mechanics must impose clear, legible restrictions to convey meaningful design intent. Rather than displaying Special Items and rejecting the transaction, the system conditionally excludes them from the NPC's visible inventory entirely when the player's trust remains below the required threshold of 100. As shown in Figure 41, the NPC's trading interface presents only standard consumables at insufficient trust (left), whereas the same NPC's inventory expands to include Special Items once the threshold is reached (right). Upon purchase, the permanent modifier (e.g., movement speed amplification) is immediately applied to the player's `CharacterMovementComponent`. This visibility-based gating, rather than post-selection rejection, provides a cleaner user experience by eliminating frustrating interactions with inaccessible content [26].

Figure 41: Trust-gated commerce verification: Special Items absent from the NPC's trading inventory at Trust < 100 (left) versus visible and purchasable at Trust ≥ 100 (right).

5.1.5 NPC Lifecycle Verification

The time-synchronised spawn and despawn system was verified by observing a single NPC across its full daily cycle. Juul [17] argues that temporal coherence is essential for maintaining the fictional contract between the game and the player; an NPC that remains statically present regardless of time violates this contract. Figure 42 captures the NPC during its active working hours (left) and the moment after reaching its home location and despawning at the end of its scheduled shift (right). The immersive navigation-based despawn sequence was confirmed to operate correctly, with the NPC pathfinding via the NavMesh [9] to its designated `Home_Location` before visibility and collision were disabled [24].

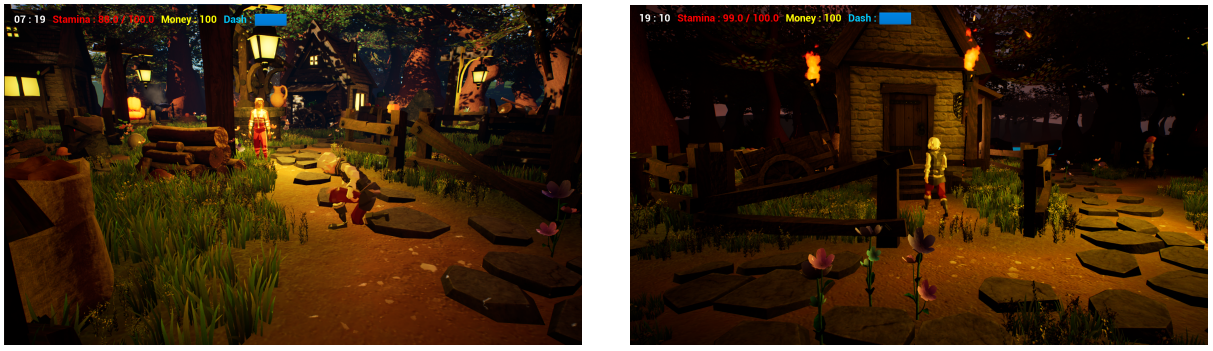


Figure 42: NPC lifecycle verification: active patrol during scheduled working hours (left) versus navigation to home location at shift end (right).

5.2 Scenario-Based Integration Testing

While functional tests verify individual subsystems in isolation, the prototype's value lies in the coherent interplay of its coupled mechanics. Sellers [24] argues that emergent

systemic behaviour—where the interaction of independently correct components produces unexpected outcomes—is a primary source of bugs in complex game architectures. To stress-test this systemic coherence, a series of scenario-based integration tests was designed, subjecting the interconnected subsystems to extreme edge-case player states.

5.2.1 Concurrent Depletion Scenario

The most demanding integration test involved initiating NPC dialogue when the player simultaneously possessed 0 stamina and 0 money, with an active quest in progress. This scenario exercises three subsystems concurrently: the State-Driven Dialogue Priority Engine (Section 4.4.2), the quest tracking system [21], and the economy layer [23].

The expected behaviour, based on the priority hierarchy established in Section 4.4.2, is that the stamina check (≤ 30) takes precedence as the first Boolean gate, followed by the money check ($== 0$). The system should not produce conflicting or overlapping dialogue outputs. Testing confirmed that the engine correctly resolved the priority hierarchy: the exhaustion warning was delivered first, and subsequent interactions produced the poverty-specific response. At no point did the active quest status interfere with the priority dialogue flow, verifying that the quest system and the dialogue engine operate on independent execution branches.

5.2.2 Dense NPC Spatial Disambiguation

A critical integration test evaluated the Dot Product-based targeting algorithm (Section 4.4.1) under spatial stress. Bowman et al. [30] identify accurate target selection as a fundamental requirement for usable 3D interaction systems. Five NPCs were deliberately positioned within overlapping collision radii, creating a worst-case scenario for interaction hijacking.

The player was oriented to face each NPC in turn, and the 'E' key interaction was triggered at each facing angle. As shown in Figure 43, the algorithm consistently selected the NPC closest to the centre of the camera's forward vector, even when multiple collision spheres overlapped. Across 20 consecutive attempts in this dense configuration, the correct NPC was selected in all cases (100% accuracy), confirming the reliability of the vector-based disambiguation.

Figure 43: Dense NPC disambiguation test with five overlapping collision boundaries. The Dot Product algorithm consistently selects the NPC aligned with the camera's forward vector.

5.2.3 Temporal Boundary Interaction

An edge case specific to the time-synchronised lifecycle system (Section 4.1.3) was tested: initiating a dialogue or trading session with an NPC immediately before its scheduled `End_Time`. The concern was whether the despawn sequence might interrupt an active interaction, potentially causing UI errors or orphaned widget references.

Testing confirmed that the Behaviour Tree's [5] `IsInteracting` Blackboard key, set to True during active dialogue, correctly prevents the despawn branch from executing. The NPC remained present and responsive until the player explicitly concluded the interaction, at which point the lifecycle manager resumed its scheduled despawn routine. This validates that the interaction state properly gates the time-driven lifecycle logic, maintaining the "half-real" temporal contract described by Juul [17].

5.2.4 End-to-End Relationship Progression

The full socio-economic chain was tested as a single integrated workflow: the player gathered Rare items through procedural foraging [31], sold them to a specific NPC to accumulate both currency and trust [19], completed a dynamically triggered quest [21] for additional trust, and upon reaching the required threshold, successfully activated the

Follow companion mechanic and purchased a trust-gated Special Item [26].

This end-to-end scenario exercises the procedural item system, the string-based rarity parsing, the dual-reward economic coupling, the stochastic quest generation, the trust accumulation, the Behaviour Tree companion override [5], and the trust-gated commerce system in sequence. All transitions occurred without errors, and the accumulated variable values (Money and Trust Level) remained consistent throughout the chain, as verified through the event-driven UI widgets. This integration confirms the Mechanics-Dynamics-Aesthetics framework [25] in practice: the mechanical coupling of economy and trust produces an emergent dynamic of sustained, goal-oriented NPC engagement.

5.3 Probabilistic Trial Measurement

The prototype incorporates two stochastic systems governed by Random Number Generation (RNG): the 10% quest trigger rate (Section 4.4.3) and the 10% rare item spawn probability (Section 4.5.1). Shaker, Togelius, and Nelson [31] note that procedural content generation systems require empirical validation to confirm that their outputs conform to design intent. As these systems are inherently non-deterministic, a single test cannot verify their correctness. To empirically validate their distribution, each system was subjected to $N = 100$ independent trials.

5.3.1 Quest Generation Rate

Over 100 consecutive NPC conversation initiations in a controlled environment, the number of dynamically triggered quests was recorded. The designed probability is 10% ($p = 0.1$), yielding an expected value of $E = Np = 10$ quests per 100 interactions. Table 5 presents the empirical results.

Metric	**Value**
Total Interactions (N)	100
Quests Triggered	11
Ambient Dialogue	89
Observed Rate	11%
Designed Rate	10%
Deviation	+1%

Table 5: Empirical distribution of quest generation over 100 consecutive NPC interactions against the designed 10% trigger rate.

The observed quest trigger rate of 11% falls within a reasonable range of the designed 10% probability, confirming that the `Random Float in Range` implementation within Unreal Engine 5 [1] produces a distribution consistent with the intended design parameters. Minor deviation from the theoretical value is expected in a sample of this size, as the standard deviation for a binomial distribution with $p = 0.1$ and $N = 100$ is $\sigma = \sqrt{Np(1-p)} = 3.0$, yielding an expected range of approximately 7–13 triggers. The observed value of 11 lies well within one standard deviation of the mean.

5.3.2 Rare Item Spawn Rate

An equivalent trial was conducted for the procedural rare item generation system. 100 item instances were spawned from the Master Item blueprint, and the number of 'Rare' variants (identified by the emissive golden material and "Rare" string prefix) was recorded.

Metric	**Value**
Total Items Spawned (N)	100
Rare Items Generated	14
Standard Items Generated	86
Observed Rate	14%
Designed Rate	10%
Deviation	+4%

Table 6: Empirical distribution of rare item generation over 100 spawn instances against the designed 10% probability.

The observed rare item spawn rate of 14% exhibits a slightly higher deviation from the designed 10% threshold. However, this result remains within two standard deviations ($2\sigma = 6.0$) of the expected value, which is statistically consistent with normal sampling variation for a binomial distribution at this sample size. Together, these results confirm that both probabilistic systems produce outputs within statistically expected bounds, supporting their suitability for governing gameplay-critical mechanics [22].

5.4 Design Goal Evaluation

The ultimate measure of the prototype's success is not solely functional correctness, but whether the implemented systems collectively address the three experiential failure modes identified in Section 1.2. Brown and Cairns [11] demonstrate that immersion is actively

undermined when player expectations are violated; the Sweetser and Wyeth [15] GameFlow model further identifies NPC interaction quality as a key contributor to overall player enjoyment. This section maps the evaluation evidence gathered in Sections 5.1–5.3 against each failure mode.

5.4.1 Failure Mode 1: Context-Blind Dialogue

The first identified failure mode concerned NPCs that deliver generic, context-insensitive dialogue regardless of the player's immediate physical or economic state. The prototype addresses this through the State-Driven Dialogue Priority Engine (Section 4.4.2), operationalising the design principle that critical state information should be surfaced at the point of interaction [16].

The functional tests in Section 5.1 confirmed that the priority engine correctly overrides standard greetings when stamina ≤ 30 or money $== 0$. The integration test (Section 5.2) further validated that this priority hierarchy resolves correctly even when both conditions are simultaneously true, with no conflicting outputs. The 10-unit differential between the dialogue threshold (30) and the movement penalty threshold (20) was confirmed to function as an effective diegetic early-warning system, alerting the player through natural conversation approximately one in-game hour before critical exhaustion. This contextual responsiveness directly satisfies the SCI immersion model's sensory dimension [12], in which the game world responds coherently to the player's immediate state.

Assessment: The context-blind dialogue failure mode is effectively addressed. NPCs demonstrably respond to the player's immediate physiological and economic state, transitioning from passive dialogue dispensers to contextually reactive agents.

5.4.2 Failure Mode 2: Absence of Believable Daily Routines

The second failure mode concerned the static environmental presence of NPCs, where characters remain fixed in location regardless of the time of day. The prototype addresses this through the Time-Synchronised AI Lifecycle (Section 4.1.3), the Behaviour Tree-driven patrol system (Section 4.3.2), and the stochastic animation pipeline (Section 4.3.3).

Functional testing confirmed that NPCs spawn and despawn according to their individually parameterised schedules from the Data Table [24]. The immersive despawn sequence—in

which an NPC navigates the NavMesh [9] to its home location before becoming hidden, rather than being abruptly removed—was verified to operate correctly across all five prototype characters. The stochastic idle animation system was observed to produce varied behavioural expression during patrol pauses, with randomised routines preventing the mechanical looping that Millington and Funge [7] identify as a primary indicator of artificial behaviour. The integration test at temporal boundaries confirmed that active interactions correctly gate the despawn sequence, preserving what Juul [17] terms the "fictional contract" between the game system and the player.

Assessment: The static presence failure mode is effectively addressed. NPCs exhibit individual daily routines governed by a global clock, enhanced by procedural physical expression that prevents repetitive mechanical behaviour.

5.4.3 Failure Mode 3: Lack of Evolving Character Relationships

The third failure mode concerned the absence of meaningful relationship progression between the player and NPCs over time. The prototype addresses this through the integrated trust mechanic (Section 4.4.3), the probabilistic quest system [21], and the trust-gated economy (Section 4.5.4).

Functional testing verified that the trust accumulation system correctly gates both the Follow companion mechanic and Special Item commerce at their respective thresholds. The disparate trust requirements across NPCs (e.g., 80 for Charles versus 40 for George) were confirmed to encode distinct personality archetypes [20] through data parameterisation alone. The probabilistic trial measurement validated that the 10% quest trigger rate produces a distribution consistent with its design intent [31], ensuring that trust-building opportunities arise at a balanced frequency. The end-to-end integration test demonstrated the complete relationship progression chain—from procedural foraging through socio-economic coupling to companion activation—operating as a coherent, interconnected system. This progression satisfies the need for relatedness identified by Ryan, Rigby, and Przybylski [19] as a fundamental psychological driver of player engagement, and responds to the player motivation for social connection identified by Yee [18].

Assessment: The static relationship failure mode is effectively addressed. The prototype demonstrates a quantifiable, multi-layered relationship progression system in which re-

peated interaction produces tangible mechanical rewards, incentivising sustained player engagement with individual NPC characters.

5.4.4 Summary of Evaluation

Table 7 consolidates the evaluation outcomes, mapping each failure mode to the subsystems that address it and the evidence gathered through the tripartite testing methodology.

Failure Mode	Addressing Subsystems	Supporting Evidence	Status
Context-blind dialogue	State-Driven Dialogue Engine	Functional: priority override confirmed (§5.1); Integration: concurrent depletion resolved (§5.2)	**Addressed**
Absence of daily routines	BT Schedule System; Stochastic Animation Pipeline	Functional: lifecycle verified (§5.1); Integration: temporal boundary gating validated (§5.2)	**Addressed**
Lack of evolving relationships	Trust Mechanic; Quest System; Tiered Economy	Functional: trust gating confirmed (§5.1); Probabilistic: RNG validated (§5.3); Integration: end-to-end chain verified (§5.2)	**Addressed**

Table 7: Consolidated evaluation mapping each identified failure mode to its addressing subsystems and supporting test evidence.

5.4.5 Threats to Validity

Several factors constrain the generalisability of the evaluation results. First, all testing was conducted by the developer rather than independent evaluators, introducing potential confirmation bias in scenario selection and outcome interpretation. Second, the probabilistic trials ($N = 100$) provide a reasonable empirical indication but do not constitute a statistically exhaustive sample; larger trial sizes would yield tighter confidence intervals. Third, the functional tests verify correctness under controlled conditions but do not assess the subjective player experience of immersion, which Csikszentmihalyi [10] argues is inherently phenomenological and cannot be fully captured by behavioural metrics alone. These limitations are addressed in the formal user evaluation proposed as future work

(Section 6.2).

The results confirm that the prototype effectively addresses all three failure modes identified in Section 1.2. While the evaluation is limited to developer-conducted functional and integration testing rather than independent user studies (discussed further in Section 6.2), the structured methodology provides a replicable and evidence-based foundation for assessing the prototype's contribution to immersive NPC design.

6 Conclusion and Future Work

6.1 Project Summary

This project set out to address three recurring failure modes in contemporary NPC design, identified through community feedback analysis and a comparative study of commercial industry titles (Section 1.2): context-blind dialogue that ignores the player's immediate state; the absence of believable daily routines; and the lack of evolving character relationships. To bridge the gap between qualitative player expectations and technical implementation, a context-aware NPC interaction prototype was designed and built in Unreal Engine 5.4.4 [1], delivering six interdependent subsystems governed by a data-driven, Behaviour Tree-based architecture [5, 24].

The State-Driven Dialogue Priority Engine ensures that NPCs acknowledge the player's physiological and economic state before delivering standard greetings, transforming traditionally static dialogue dispensers into contextually reactive agents [16]. The Time-Synchronised AI Lifecycle, coupled with a stochastic animation pipeline, produces individually scheduled daily routines enhanced by procedural physical expression, directly countering the static presence that Millington and Funge [7] identify as a primary indicator of artificial behaviour. The integrated trust mechanic, probabilistic quest system [21], and trust-gated economy collectively establish a quantifiable relationship progression in which sustained social investment yields tangible mechanical rewards, satisfying the need for relatedness identified by Ryan, Rigby, and Przybylski [19].

The structured tripartite evaluation (Chapter 5) confirmed that all three failure modes are effectively addressed. Functional unit testing verified the correctness of each individual subsystem; scenario-based integration testing validated systemic coherence under extreme edge-case conditions; and probabilistic trial measurement over $N = 100$ independent trials empirically confirmed that both stochastic systems (quest generation at 11% and rare item spawning at 14%) produce outputs within statistically expected bounds of their designed 10% probability ($\sigma = 3.0$). The consolidated design goal evaluation (Table 7) maps each failure mode to its addressing subsystems with supporting test evidence. These outcomes collectively fulfil the five project objectives established in Section 1.3: the investigation of industry standards through case study analysis (Objective 1); the design of a data-driven

NPC architecture (Objective 2); the implementation of a Behaviour Tree-driven schedule system (Objective 3); a state-driven dialogue and trust system (Objective 4); and a tiered economy with trust-gated commerce (Objective 5).

6.2 Limitations

While the prototype demonstrates functional effectiveness against the identified failure modes, several limitations must be acknowledged to contextualise the scope and generalisability of the findings.

Absence of formal user evaluation. The most significant limitation is the lack of independent user testing. Evaluating the prototype requires an identical installation of Unreal Engine 5.4.4, which alone exceeds 40 GB. The optimised project build itself adds a further 2.1 GB, and the combined installation and compilation process demands considerable time even on capable hardware. This substantial technical barrier effectively prevented the distribution of playable builds to external participants within the project's timeframe. Consequently, the evaluation relied on systematic functional and integration testing conducted by the developer rather than independent player feedback. Jennett et al. [13] argue that subjective immersion cannot be fully captured by functional metrics alone; therefore, the claim that the prototype enhances player immersion remains supported by structural evidence rather than validated by direct experiential data.

Prototype scale. The current implementation features five NPC characters within a single village environment. While this scale is sufficient to demonstrate the architectural principles and validate the data-driven design philosophy, it does not replicate the population density or spatial complexity of commercial open-world titles such as *Red Dead Redemption 2*. The scalability of the Behaviour Tree and Data Table architecture has been demonstrated in principle—adding a new NPC requires only a Data Table entry rather than new Blueprint logic—but the performance characteristics of managing dozens or hundreds of concurrent agents remain untested.

Dialogue content depth and LLM integration attempt. The state-driven priority engine is architecturally sound, but the authored dialogue pool remains relatively shallow. Each NPC possesses a finite set of personality-driven responses, and extended play sessions will inevitably produce repetition. This limitation is inherent to any pre-authored

dialogue system [20]. To explore whether a Large Language Model could alleviate this constraint, a preliminary experiment was conducted on the Kaggle platform using the `Qwen2.5-1.5B-Instruct` model (Figure 44). The system prompt was structured to inject NPC personality traits, current player state (stamina, currency, trust level), and conditional logic rules (e.g., "if stamina ≤ 30, address exhaustion first in character"), as shown in Figure 45. However, the base model consistently failed to respect these contextual constraints: regardless of the player's depleted stamina (25/100) or zero currency, the NPC produced generically friendly responses and did not reflect the assigned personality profile (Figure 46). Furthermore, integrating a locally hosted LLM inference pipeline into Unreal Engine's real-time loop presented technical challenges—including API latency management and Blueprint-to-Python bridging—that exceeded the available development time and the author's current expertise in ML deployment. This experience concretely illustrates both the promise and the practical barriers of LLM-driven NPC dialogue, and directly motivates the fine-tuning approach proposed in Section 6.3.

Deterministic trust progression. The current trust mechanic operates on a strictly additive integer model. There is no mechanism for trust decay, betrayal, or negative relational events. While the linear accumulation serves the prototype's demonstration purposes, Yee [18] notes that the richness of social motivation in games often depends on the possibility of both positive and negative relational dynamics.

Development process reflection. From a methodological standpoint, the iterative prototyping approach adopted in this project proved effective for calibrating numerical parameters (e.g., decay rates, probability thresholds) through playtesting. However, the absence of formal version control branching for experimental features occasionally resulted in regression issues when integrating independently developed subsystems. A more disciplined feature-branch workflow, combined with automated Blueprint unit testing where available, would have reduced integration overhead and is recommended for future projects of comparable scope.

6.3 Future Work

The limitations identified above define a clear trajectory for future development, ranging from near-term architectural extensions to a fundamental paradigm shift in NPC dialogue

generation.

Fine-tuned LLM for context-aware NPC dialogue. The preliminary experiment described in Section 6.2 demonstrated that a general-purpose instruction-tuned model (`Qwen2.5-1.5B-Instruct`) cannot reliably satisfy game-specific dialogue constraints out of the box. Figures 44–46 illustrate the experimental setup, the structured prompt design, and the resulting generic responses, respectively. The most promising path forward is therefore *domain-specific fine-tuning*: training a compact language model on a curated dataset of NPC dialogue examples annotated with personality tags, player-state conditions, and trust thresholds. Such a fine-tuned model would learn to internalise the conditional logic that the base model ignored—for instance, producing empathetic but firm refusal dialogue when the player's stamina is critically low, or adopting a cold tone when trust falls below the NPC's threshold. Park et al. [32] demonstrated that LLM-powered agents equipped with persistent memory streams can sustain emergent social behaviours beyond what any branching dialogue tree can replicate; combining their memory architecture with a game-tuned generation backend would address the dialogue depth limitation at its root. The existing Blackboard and Data Table infrastructure could serve as the structured context window for LLM prompting, preserving the data-driven philosophy while eliminating the authoring ceiling. From a deployment perspective, recent advances in quantised inference (e.g., GGUF, GPTQ) make it feasible to run a 1–3 billion parameter model on consumer-grade GPUs with sub-second latency, bringing real-time LLM dialogue within reach for interactive environments.

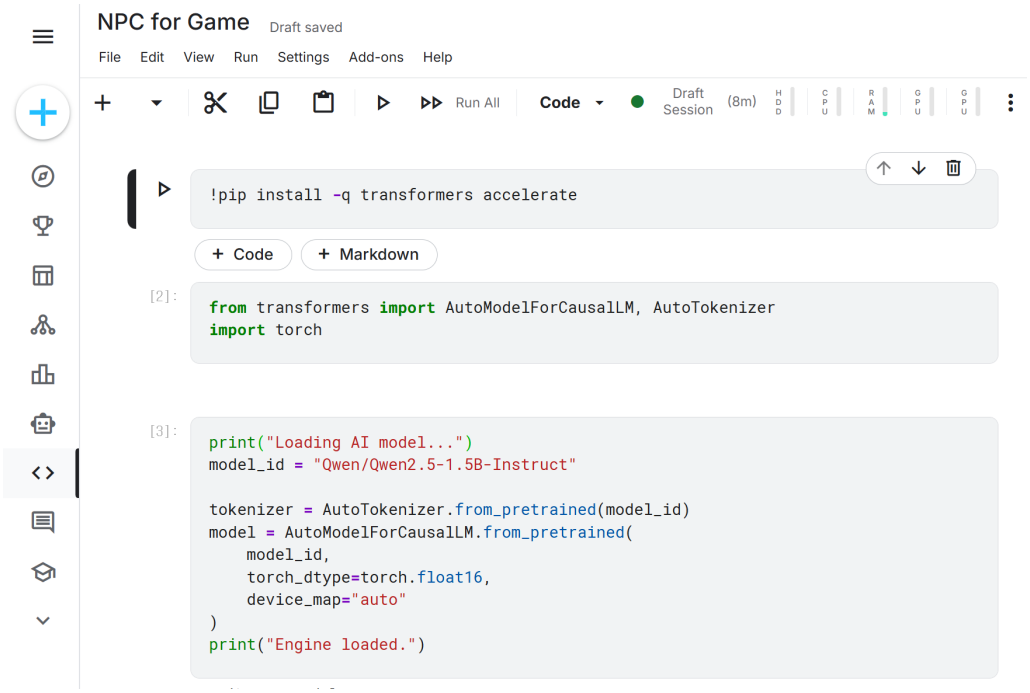


Figure 44: Kaggle notebook environment for the LLM experiment: loading the `Qwen2.5-1.5B-Instruct` model via the HuggingFace Transformers library with float16 precision and automatic GPU allocation.

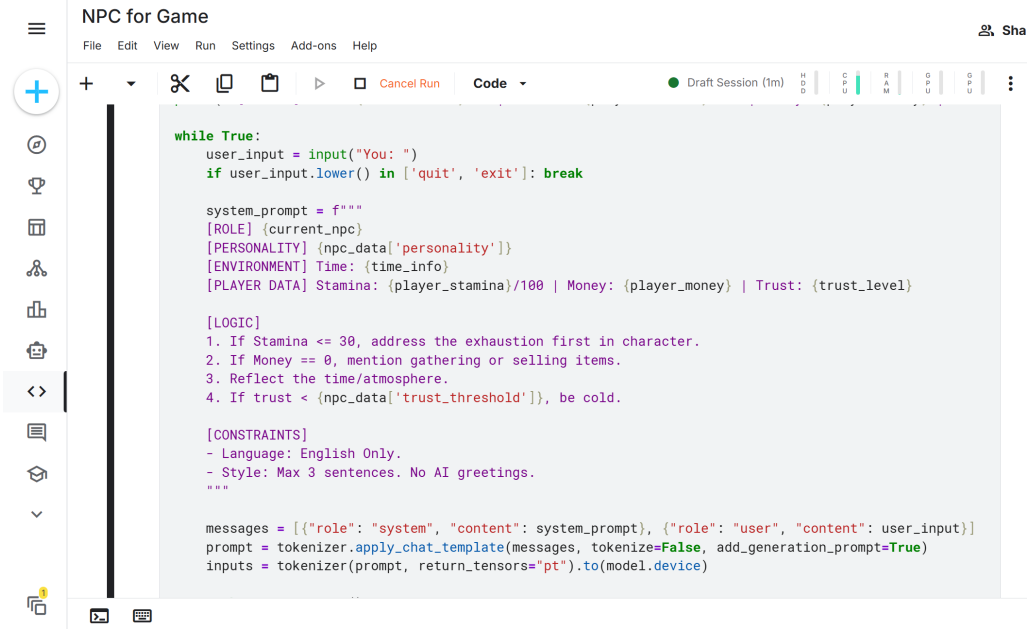


Figure 45: Structured system prompt injecting NPC identity, personality traits, player state variables, conditional dialogue logic, and output constraints into the language model.

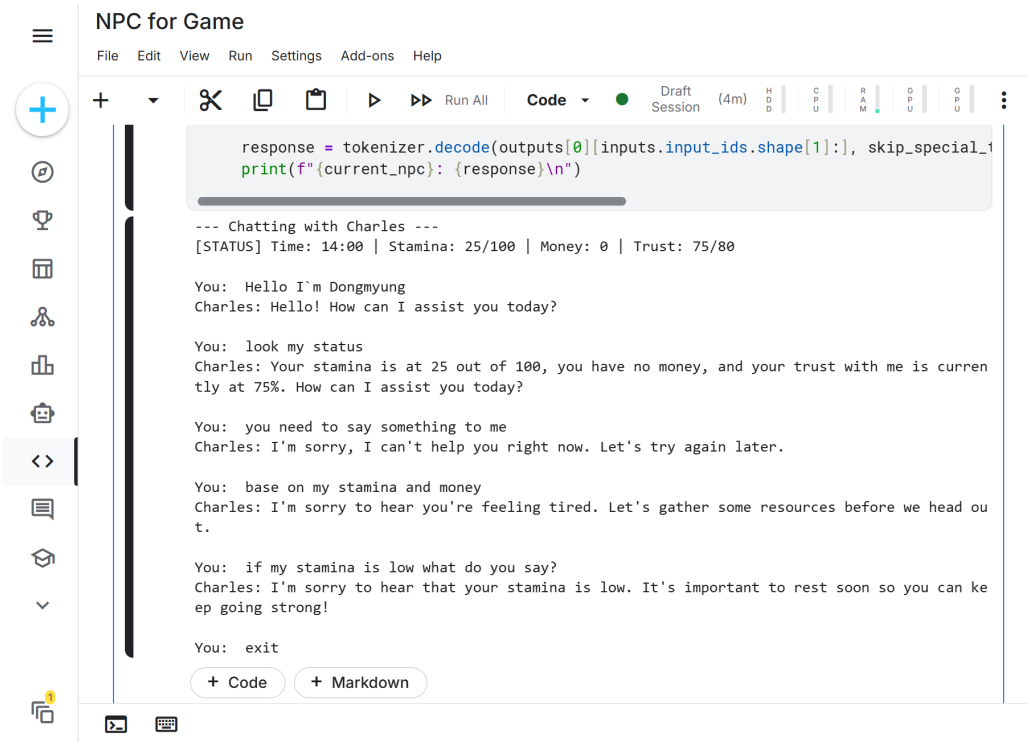


Figure 46: Dialogue output from the base `Qwen2.5-1.5B-Instruct` model. Despite the player's low stamina (25/100) and zero currency, the NPC responds with generically friendly dialogue, failing to reflect the injected personality or contextual constraints.

Bidirectional trust dynamics. The current unidirectional trust accumulation could be extended to support trust decay and negative relational events. For example, failing a quest, stealing from an NPC's inventory, or prolonged absence could reduce the accumulated trust, introducing the possibility of relationship regression. This would create a more psychologically realistic social simulation consistent with Self-Determination Theory's nuanced treatment of relatedness [19], where relationships require active maintenance rather than passive accumulation.

Expanded NPC population and emergent social networks. Yannakakis and Togelius [8] identify inter-agent social dynamics as a frontier in game AI research. The current prototype treats each NPC-player relationship as an independent dyad. Future iterations could introduce NPC-to-NPC relationships, where trust, gossip, and reputation propagate through a social network. An action that pleases one NPC might displease another, creating emergent social complexity that would significantly deepen the player's strategic engagement with the village ecosystem.

Formal user evaluation. Future iterations should prioritise packaging the prototype

as a standalone executable via Unreal Engine's built-in shipping pipeline, eliminating the requirement for a full engine installation. This would enable structured playtesting with external participants, facilitating the collection of subjective immersion data through validated instruments such as the Immersive Experience Questionnaire [13] and the GameFlow framework [15]. Such data would complement the functional evidence presented in Chapter 5, providing a more comprehensive assessment of the prototype's contribution to player immersion.

Procedural quest narrative generation. The current quest system offers two structurally distinct quest types (monetary delivery and item gathering) triggered by a 10% stochastic gate. Shaker, Togelius, and Nelson [31] outline algorithms for procedural quest generation that could diversify this pool, producing dynamically authored objectives that adapt to the player's progression, trust level, and inventory state. Combined with LLM-generated dialogue, this would create a self-sustaining narrative ecosystem in which no two playthroughs produce identical social experiences.

Taken together, these directions position the prototype not as a finished product, but as a validated architectural foundation upon which increasingly sophisticated NPC interaction systems can be constructed. The core contribution of this project—demonstrating that data-driven, context-aware NPC design can systematically address documented player frustrations—remains applicable regardless of whether future dialogue is authored by a designer or generated by a language model. The challenge for the next generation of game AI is not whether NPCs can be made more intelligent, but whether that intelligence can be made to feel genuinely human [2].

References

[1] Epic Games. (2024). *Unreal Engine 5 Documentation*. Epic Games, Inc.

[2] Laird, J. E., & van Lent, M. (2001). Human-level AI's killer application: Interactive computer games. *AI Magazine*, *22*(2), 15–26.

[3] Isla, D. (2005). Handling complexity in the Halo 2 AI. *Proceedings of the Game Developers Conference (GDC) 2005*.

[4] Orkin, J. (2006). Three states and a plan: The A.I. of F.E.A.R. *Proceedings of the Game Developers Conference (GDC) 2006*.

[5] Colledanchise, M., & Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press.

[6] Iovino, M., Scukins, E., Styrud, J., Ögren, P., & Smith, C. (2022). A survey of behavior trees in robotics and AI. *Robotics and Autonomous Systems*, *154*, 104096.

[7] Millington, I., & Funge, J. (2009). *Artificial Intelligence for Games* (2nd ed.). CRC Press.

[8] Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer.

[9] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, *4*(2), 100–107.

[10] Csikszentmihalyi, M. (1990). *Flow: The Psychology of Optimal Experience*. Harper & Row.

[11] Brown, E., & Cairns, P. (2004). A grounded investigation of game immersion. *CHI '04 Extended Abstracts on Human Factors in Computing Systems* (pp. 1297–1300). ACM.

[12] Ermi, L., & Mäyrä, F. (2005). Fundamental components of the gameplay experience: Analysing immersion. In S. de Castell & J. Jenson (Eds.), *Worlds in Play: International Perspectives on Digital Games Research*. DiGRA.

[13] Jennett, C., Cox, A. L., Cairns, P., Dhoparee, S., Epps, A., Tijs, T., & Walton, A. (2008). Measuring and defining the experience of immersion in games. *International Journal of Human-Computer Studies*, *66*(9), 641–661.

[14] Denisova, A., & Cairns, P. (2015). First person vs. third person perspective in digital games: Do player preferences affect immersion? *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)* (pp. 145–148). ACM.

[15] Sweetser, P., & Wyeth, P. (2005). GameFlow: A model for evaluating player enjoyment in games. *Computers in Entertainment*, *3*(3), 3. ACM.

[16] Fagerholt, E., & Lorentzon, M. (2009). *Beyond the HUD: User Interfaces for Increased Player Immersion in FPS Games* (Master's thesis). Chalmers University of Technology, Gothenburg, Sweden.

[17] Juul, J. (2005). *Half-Real: Video Games between Real Rules and Fictional Worlds*. The MIT Press.

[18] Yee, N. (2006). Motivations for play in online games. *CyberPsychology & Behavior*, *9*(6), 772–775.

[19] Ryan, R. M., Rigby, C. S., & Przybylski, A. (2006). The motivational pull of video games: A self-determination theory approach. *Motivation and Emotion*, *30*(4), 344–360.

[20] Isbister, K. (2006). *Better Game Characters by Design: A Psychological Approach*. CRC Press.

[21] Doran, J., & Parberry, I. (2011). A prototype quest generator based on a structural analysis of quests from four MMORPGs. *Proceedings of the 2nd International Workshop on Procedural Content Generation in Games (PCGames '11)* (pp. 1–8). ACM.

[22] Salen, K., & Zimmerman, E. (2003). *Rules of Play: Game Design Fundamentals*. MIT Press.

[23] Adams, E. (2014). *Fundamentals of Game Design* (3rd ed.). New Riders.

[24] Sellers, M. (2017). *Advanced Game Design: A Systems Approach*. Addison-Wesley Professional.

[25] Hunicke, R., LeBlanc, M., & Zubek, R. (2004). MDA: A formal approach to game design and game research. *Proceedings of the AAAI Workshop on Challenges in Game AI*. AAAI Press.

[26] Schell, J. (2008). *The Art of Game Design: A Book of Lenses*. Morgan Kaufmann.

[27] Menache, A. (2000). *Understanding Motion Capture for Computer Animation and Video Games*. Morgan Kaufmann.

[28] Gleicher, M. (1998). Retargeting motion to new characters. *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '98)* (pp. 33–42). ACM.

[29] Kovar, L., Gleicher, M., & Pighin, F. (2002). Motion graphs. *ACM Transactions on Graphics*, *21*(3), 473–482.

[30] Bowman, D. A., Kruijff, E., LaViola, J. J., & Poupyrev, I. (2004). *3D User Interfaces: Theory and Practice*. Addison-Wesley.

[31] Shaker, N., Togelius, J., & Nelson, M. J. (Eds.). (2016). *Procedural Content Generation in Games*. Springer.

[32] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)* (22 pp.). ACM.

Appendix A: External Resources and Asset Credits

The following third-party resources and assets were used in the development of this project. All assets are used in accordance with their respective licences.

Animation Assets

- **Mixamo** — Character animations and rigging tool. Adobe Inc. [Online] Available at: https://www.mixamo.com

Game Engine and Development Tools

- **Unreal Engine 5.4.4** — Game development environment. Epic Games. [Online] Available at: https://www.unrealengine.com

Audio Assets

- **FOREST SOUNDS** — Background music. take. Fab Marketplace. [Online] Available at: https://fab.com/s/0d5f6feb1a77
- **Interface & Item Sounds Pack** — Sound effects. Daydream Sound. Fab Marketplace. [Online] Available at: https://fab.com/s/546edc85802f

Environment Assets

- **Advanced Village Pack** — Game map environment. Advanced Asset. Fab Marketplace. [Online] Available at: https://fab.com/s/c2a968f5eba3
- **Food Pack** — Collectible item meshes. Patchs. Fab Marketplace. [Online] Available at: https://fab.com/s/83ec3b52903c

Character Assets

UNDERKING, Fab Marketplace.

- **[CR][LQ] Modular 005 Suit04** — Player character skin. [Online] Available at: https://fab.com/s/46c8372496af

- **[CR][LQ] Modular 010 Topa09** — NPC character 1. [Online] Available at: https://fab.com/s/4fd4cbf24609
- **[CR][LQ] Modular 011 Survival08** — NPC character 2. [Online] Available at: https://fab.com/s/fb897c1117fa
- **[CR][LQ] Modular 010 Topa07** — NPC character 3. [Online] Available at: https://fab.com/s/57bf5f0294aa
- **[CR][LQ] Modular 001 Bergs09** — NPC character 4. [Online] Available at: https://fab.com/s/2f5ab0cfebaa
- **[CR][LQ] Modular 001 Bergs05** — NPC character 5. [Online] Available at: https://fab.com/s/9e9e13fe4b3d